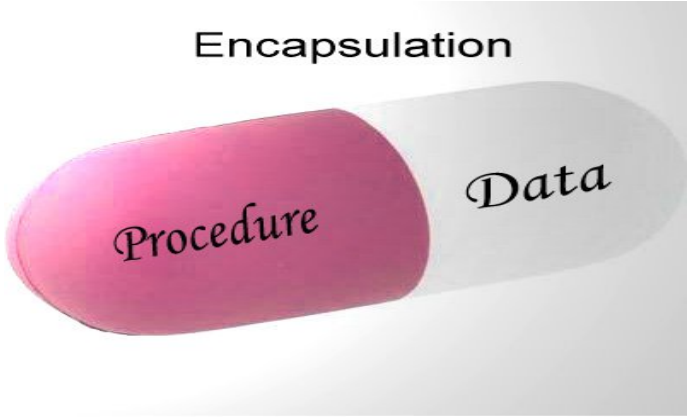
OOAD



- **Encapsulation**
  - **Packaging of several items together into one unit (both attributes and behavior of the object), Also protects the contents.**
  - **The only way to access or change an object's attribute is through that object's specific behavior.**
  - **Objects *encapsulates* what they do.**
    - That is, they hide the inner workings of their operations
      - **from the outside world**
      - **and from other objects**

Eg. Class BankAcount
{
Private:
Balance
……
Withdraw(int)

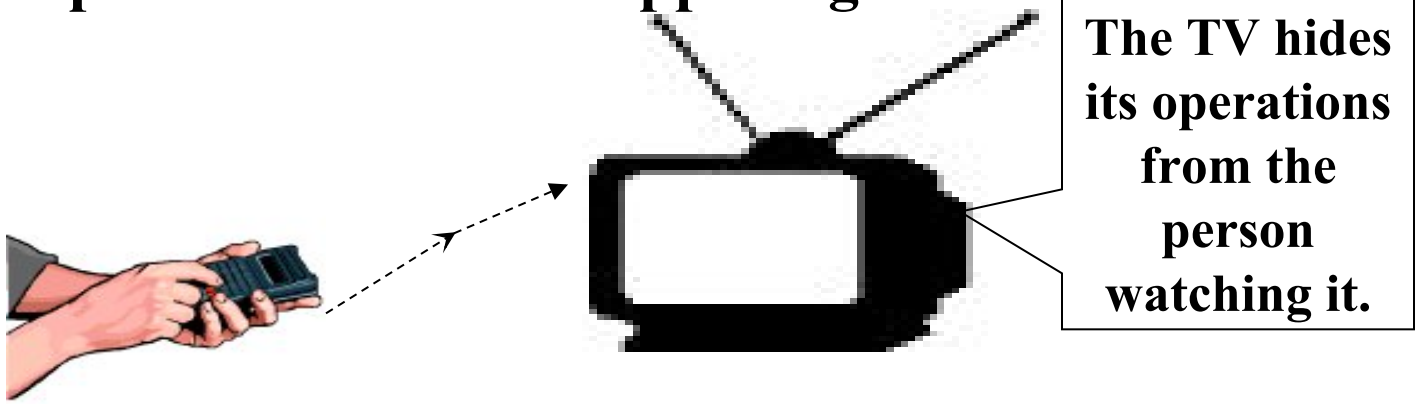
- Encapsulation

  When an object carries out its operations, those operations are hidden.

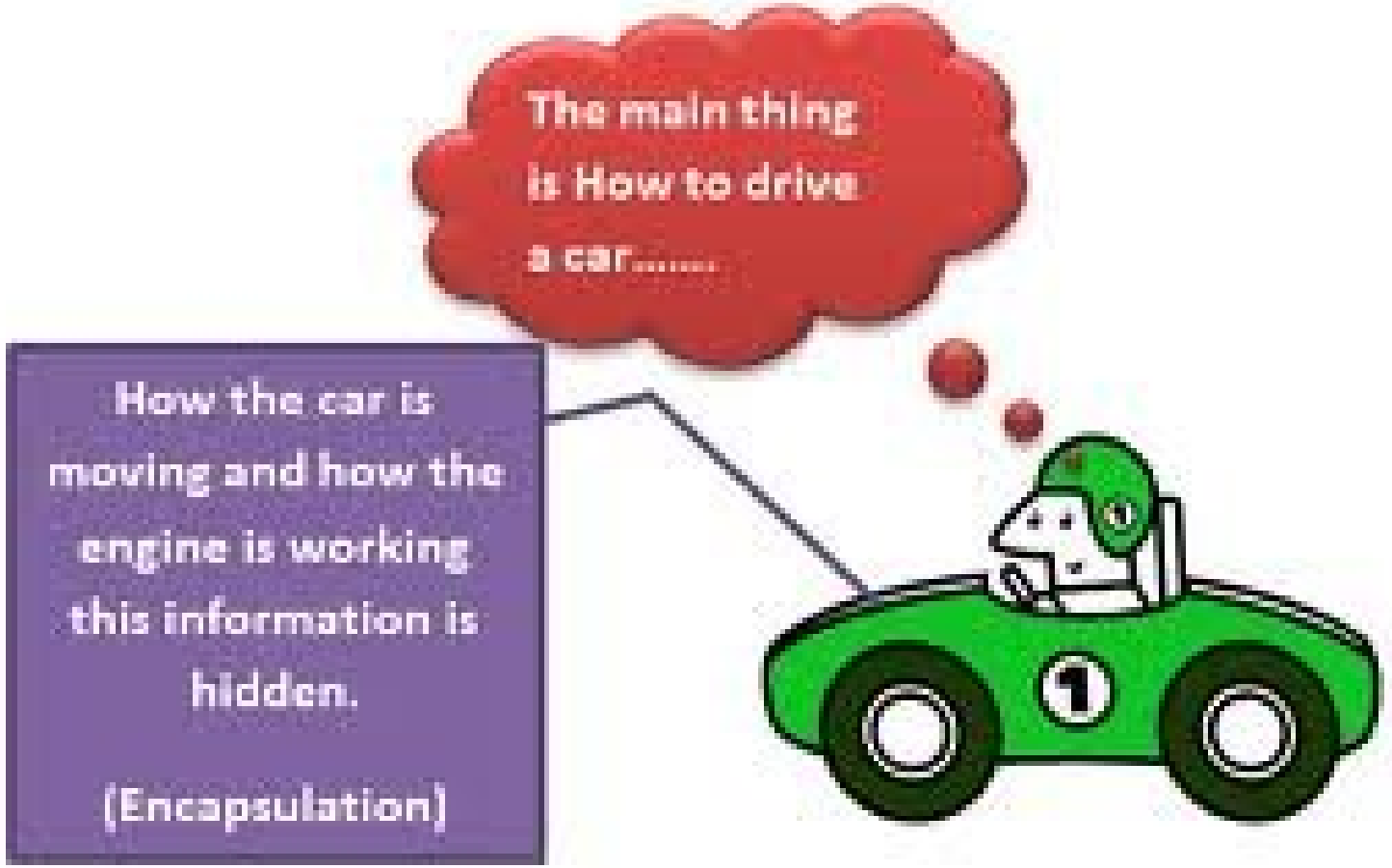
E.g. When most people watch a television show,

- they usually don't know or care about the complex electronics that sit in back of the TV screen

- or the operations that are happening.



System concepts for object modeling
Encapsulation



- **Encapsulation**

  Why is this important?

  - In the software world, encapsulation helps cut down on the potential for bad things to happen.
  - In a system that consists of objects, the object depends on each other in various ways.
  - If one of them happen to malfunction, software engineers have to change it in some way.
  - Hiding its operations from other objects means it probably won't be necessary to change those other objects.

- Encapsulation
  - Turning from software to reality, you see the importance of *encapsulation*.

eg. Your computer monitor, hides its operations from your computers CPU.

When something goes wrong with your monitor, you either fix or replace it.

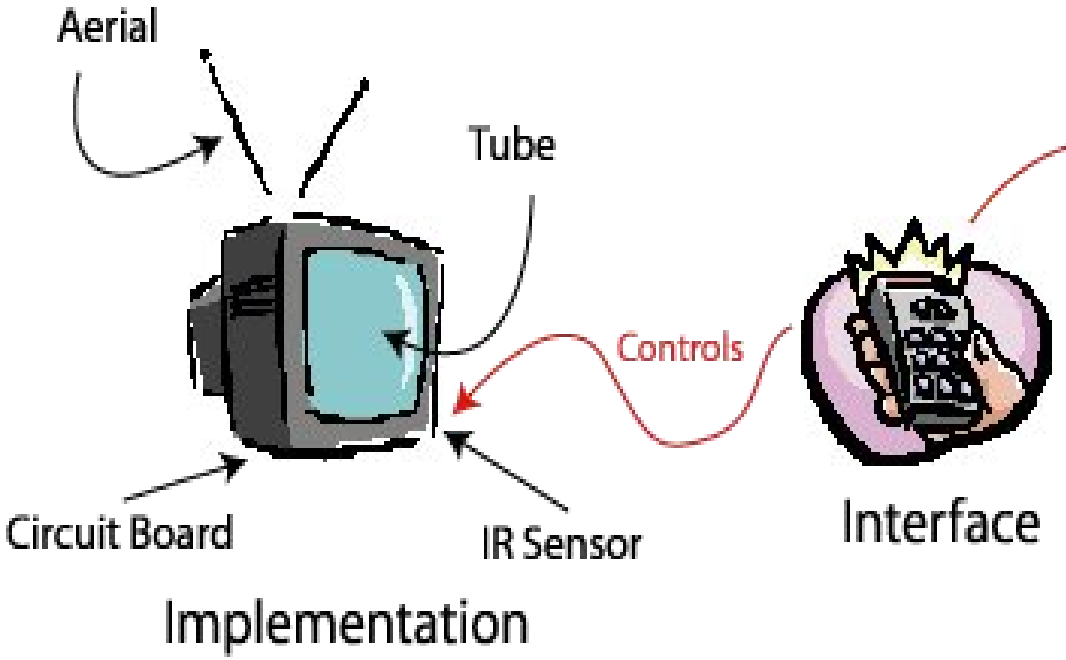
You probably won’t have to fix or replace the CPU along with it.

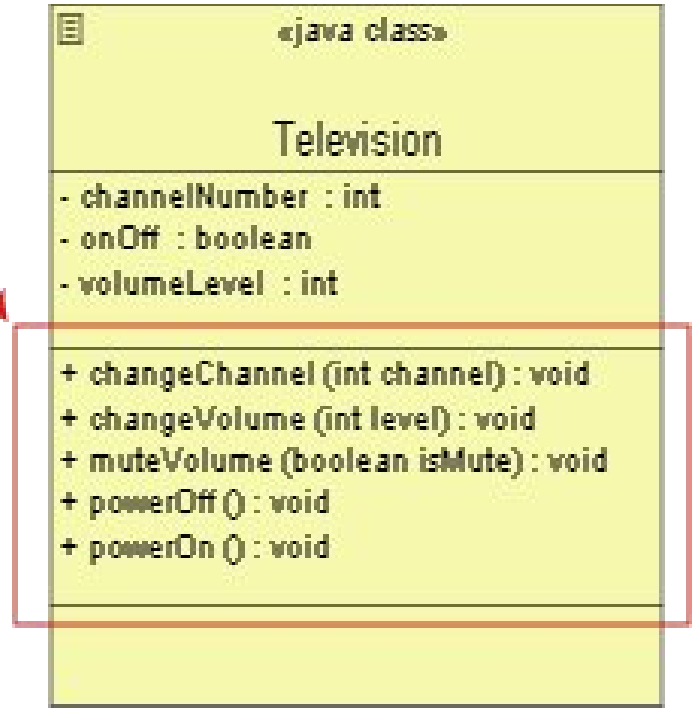
- Encapsulation
  - An object hides what it does from other objects and from outside world
  - But an object does have to present a “face“ to the outside world, so you can initiate those operations.

    eg. A TV, has a set of buttons either on the TV or on a remote.

    The TV’s buttons are called *interfaces*.

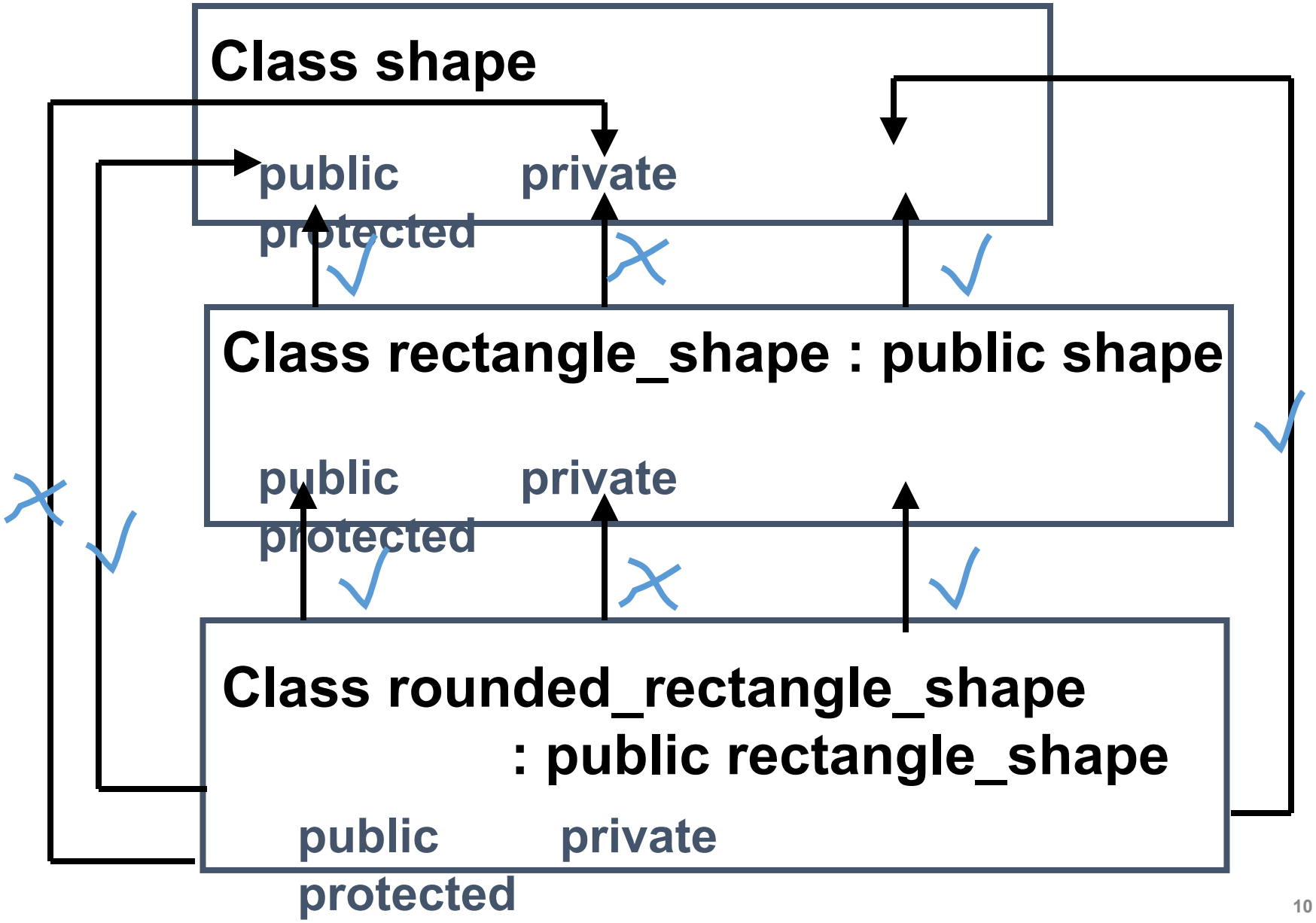
System concepts for object modeling
Encapsulation





- Encapsulation
  - Encapsulation is done through the definition of *region of access*.
  - *Region of access* defines the accessibility of the *services* or *attributes* in a class.
  - In C++ (OO programming Language)
    - 3 types of access regions: *private*, *public*, *protected*
  - *In Java*
    - *Access modifiers: default, public, private, protected,*
    - *Static modifier, final modifier, synchronized modifier, native modifier*

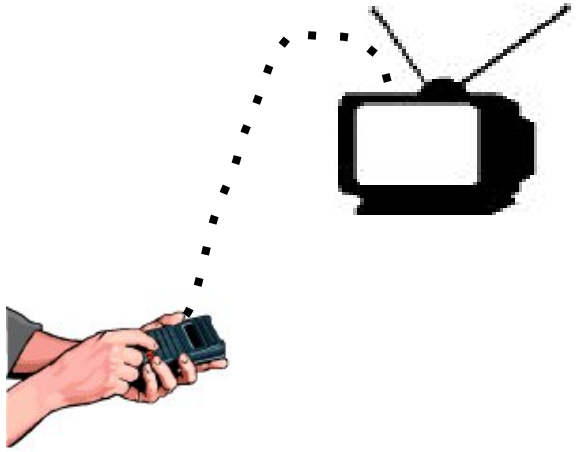
Access to the Base Class



- Message Passing
  - In a system, objects work together.
  - They do this by sending messages to one another.
  - One object sends another a message to perform an operation.
  - The receiving object performs that operation.

eg. A TV and a Remote

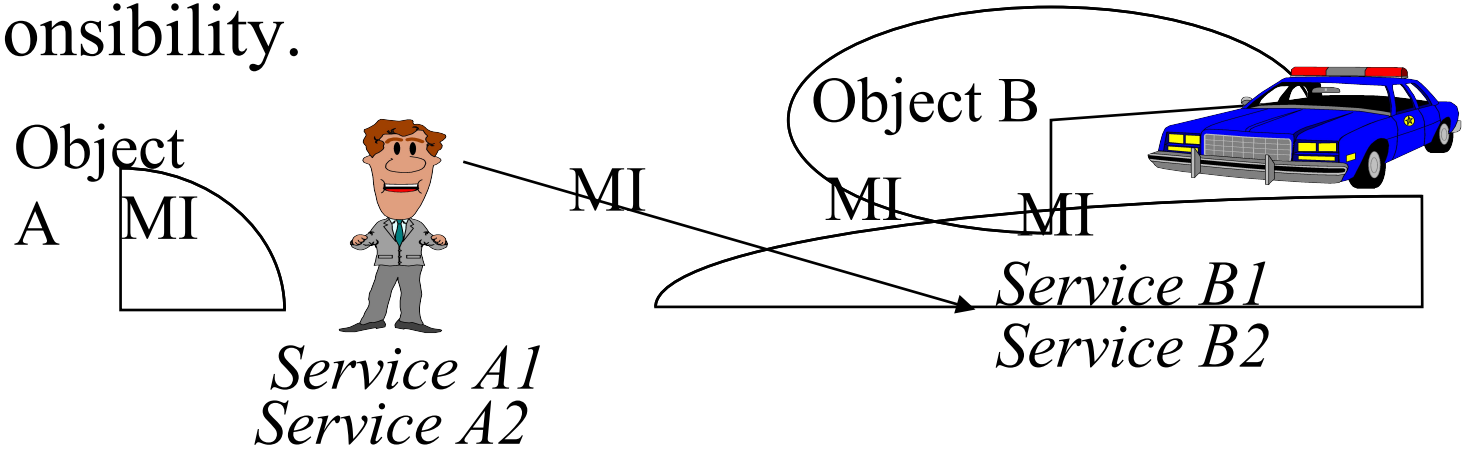
- Message Passing
  - When you want to watch a TV show,
    - You hunt round for a remote,
    - Settle into your favorite chair and
    - Push the *On* Button.
  - What happens?
    - The remote object sends a message (literally!) to the TV object to turn itself on.
    - The TV object receives the message.
    - It knows how to perform the turn-on operation, and turns itself on.



- Message Passing
  - When you want to watch a different channel,
    - You click the appropriate button on the remote,
    - Remote object sends a different message *change channel* to the TV object.
  - The remote can also communicate with the TV via other messages for *changing the volume*, *muting the volume etc.*.

- Message Passing
  - Let’s go back to interfaces for a moment.
  - Most of the things you do from the remote, you can also do by *getting out of the chair*, *going to the TV*, and *clicking buttons on the TV*.
  - The interface the TV presents to you is obviously not the same interface it presents to the remote.

- Each object offers services within the realm of its responsibility.



* Objects use other objects services through *<u>message passing</u>*

* An object may even call its own services

- Relationships
  - A natural business association that exists between one or more objects and classes

    e.g. You interact with a text book by reading it, with a telephone by using it, People interact with each other by communicating with them.

- Association
  - When you turn on your TV, in object oriented terms, you are in an *association* with your TV.
  - An association is unidirectional (one way) or bi-directional (two way).

    eg. *is married to*
  - Some times an object might be associated with another in more than one way.

    Gihan *is a co-worker of* Damith

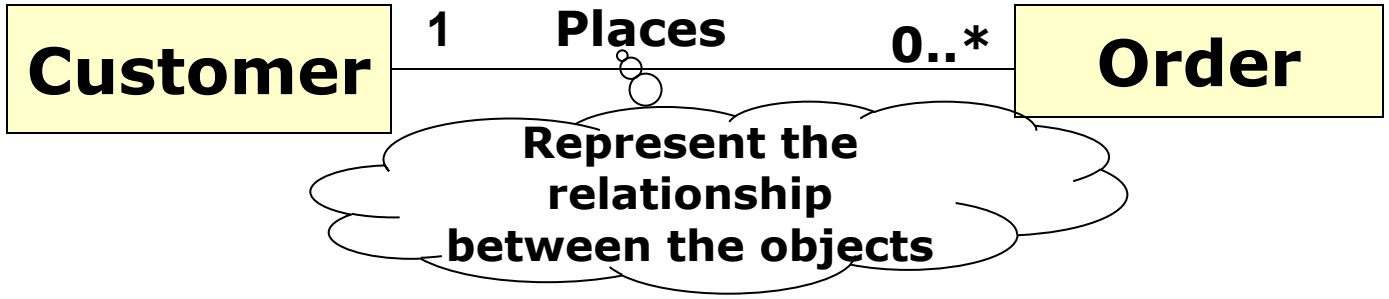
    Gihan *is a friend of* Damith

- Association

  e.g.

  A CUSTOMER PLACES zero or more ORDERS

  An ORDER IS PLACED BY one and only one CUSTOMER



UML Multiplicity Notations

Multiplicity – the minimum and maximum number of occurrences of one object/class for a single occurrence of the related object/class.

Multiplicity	UML Multiplicity Notation	Association with Multiplicity	Association Meaning
Exactly 1	1 or *leave blank*	Employee — Works for 1 — Department Employee — Works for — Department	An employee works for one and only one department.
Zero or 1	0..1	Employee — Has 0..1 — Spouse	An employee has either one or no spouse.
Zero or more	0..* or *	Customer — Makes 0..* — Payment Customer — Makes * — Payment	A customer can make no payment up to many payments.
1 or more	1..*	University — Offers 1..* — Course	A university offers at least 1 course up to many courses.
Specific range	7..9	Team — Has scheduled 7..9 — Game	A team has either 7, 8, or 9 games scheduled

Introduction to Object Modelling

- Object oriented languages (Java & C++,C#) are growing in popularity.
  - Because object oriented programming
    - can **promote better code reuse** to hold down program costs
    - More appropriate for projects where
      - geographically separated groups of programmers have to collaborate to produce an integrated system and
      - each team can develop pieces of programming code independently

Introduction to Object Modelling

Object-oriented analysis (OOA) – an approach used to

1. study existing **objects** to see if they can be reused or adapted for new uses
2. define new or modified objects that will be combined with existing objects into a useful business computing application

Object modeling – a technique for identifying objects within the systems environment and the relationships between those objects.

Introduction to Object Modelling

- Some of the object oriented diagrams, such as **class** and **sequence** diagrams would be inappropriate except when the system will be implemented in an OO environment
- Other diagrams developed for OOAD can be used in any kind of environment
  - e.g. **Use cases**

Introduction to Unified Modeling Language (UML)

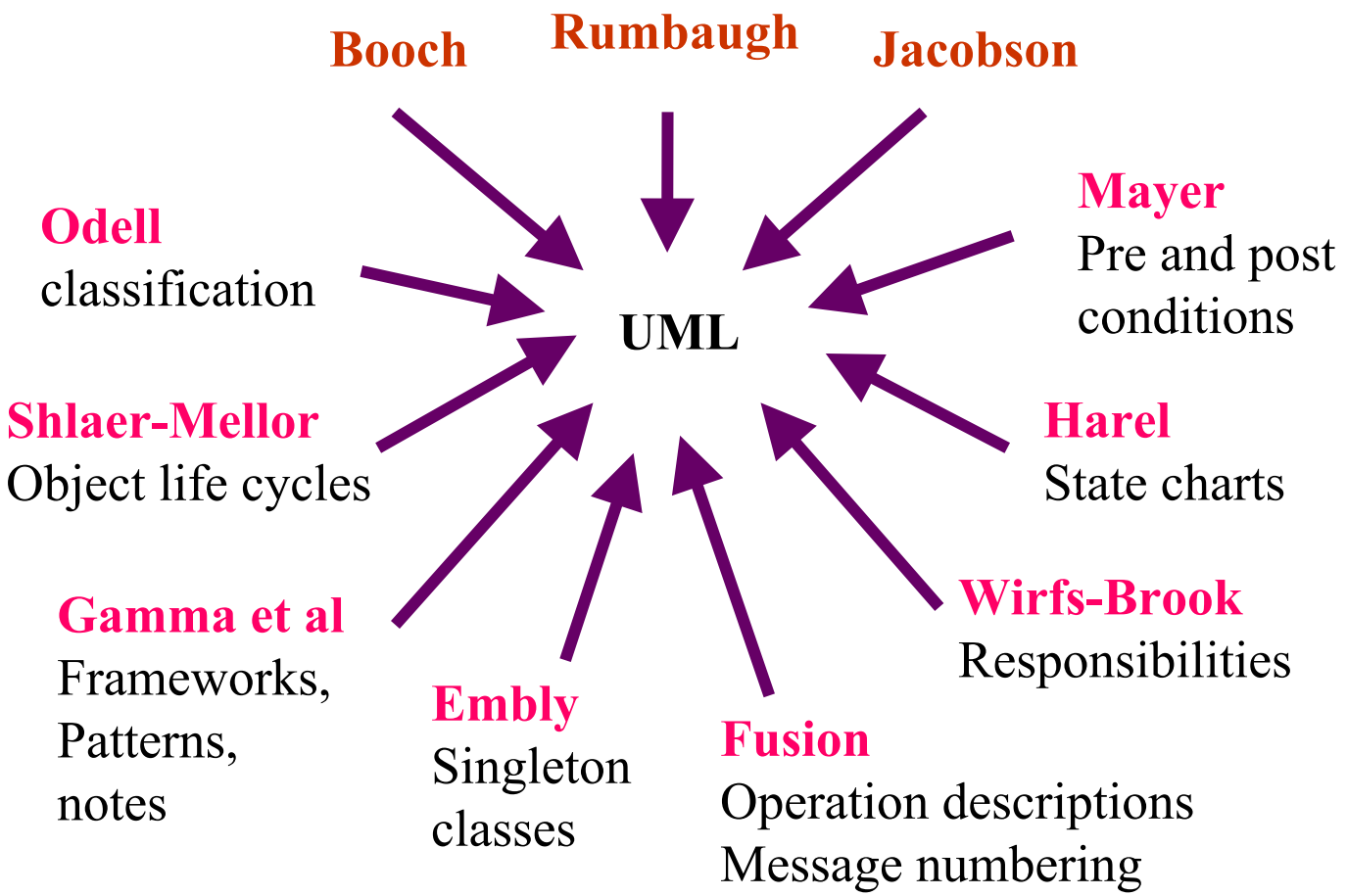
- One of the most exciting and useful tools in the world of system development
- A visual modeling language
- Enables system builders to create blueprints that capture their visions in a standard, easy-to-understand way
- Provides a mechanism to effectively share and communicate these visions with others (Technical or Non technical)

Introduction to Unified Modeling Language (UML)

- It most directly unifies the methods of
  - Booch,
  - Rumbaugh (OMT) and
  - Jacobson

as well as the best ideas from a number of other methodologies.

UML Inputs


Booch
Rumbaugh
Jacobson
Odell
classification
Mayer
Pre and post
conditions
UML
Shlaer-Mellor
Object life cycles
Harel
State charts
Gamma et al
Frameworks,
Patterns,
notes
Wirfs-Brook
Responsibilities
Embly
Singleton
classes
Fusion
Operation descriptions
Message numbering

Introduction to the UML

Unified Modeling Language (UML) – a set of modeling conventions that is used to specify or describe a software system in terms of objects.

- The UML does **not prescribe a method** for developing systems—**only a notation** that is now widely accepted as a standard for object modeling.

UML History

- The first public draft version – (version 0.8) – Oct 1995
- Feedback from public and Ivor Jacobson's inputs included – (ver. 0.9 Jul 1996, ver. 0.91 Oct 1996)
- Ver 1.0/1.1 was presented to OMG group for standardization in July/Sep 1997.
- Nov 1997 – UML adopted as the standard modelling language by OMG
- Ver 1.2 - June 1998
- Ver 1.3 – Dec 1998
- Ver 1.4 – 2000
- Ver 2.0 – 2003
- Ver 2.1 – 2007
- Ver 2.2 - 2009
- Ver 2.3 – 2010,
- Ver 2.4 –Jan 2011
- Ver 2.5 June 2015

UML
Components and Capabilities

- The UML consists of a number of graphical elements that combine to form diagrams.
- Because it is a language, the UML has rules for combining these elements.
- Lets look at the diagrams before looking at the elements and rules. You will be using these diagrams to do systems analysis and design.
- The purpose of the diagrams is to present multiple views (model) of a system.
- UML 2.* include many different diagrams.

UML Diagrams

- UML has 14 types of diagrams divided into two categories.

- Structure Diagrams :

Seven diagram types represent *structural* information,

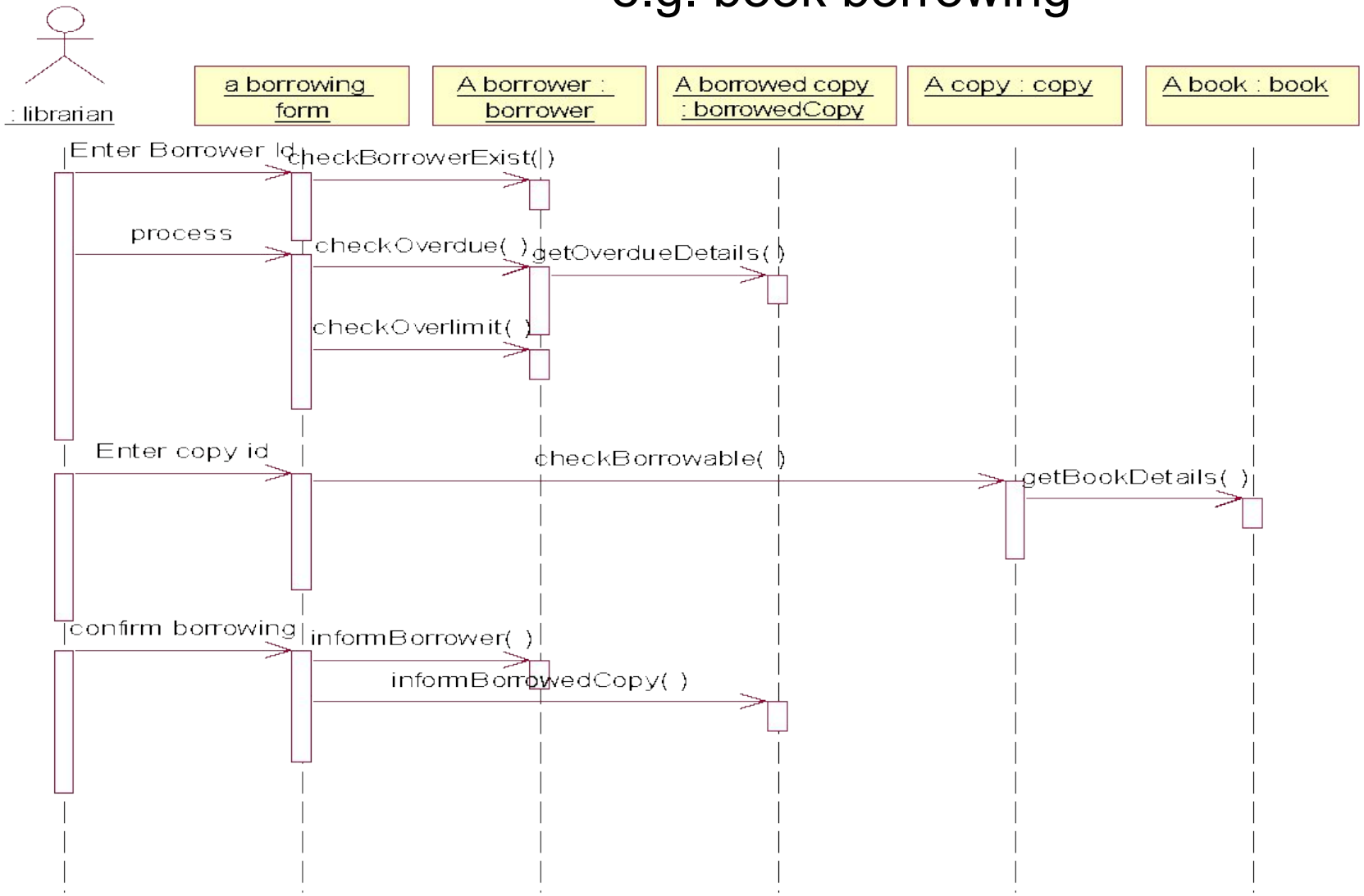
-Behaviour Diagrams :

other seven represent general types of *behavior*..

UML Diagrams

Sequence Diagrams

e.g. book borrowing



UML Diagrams

Profile Diagram

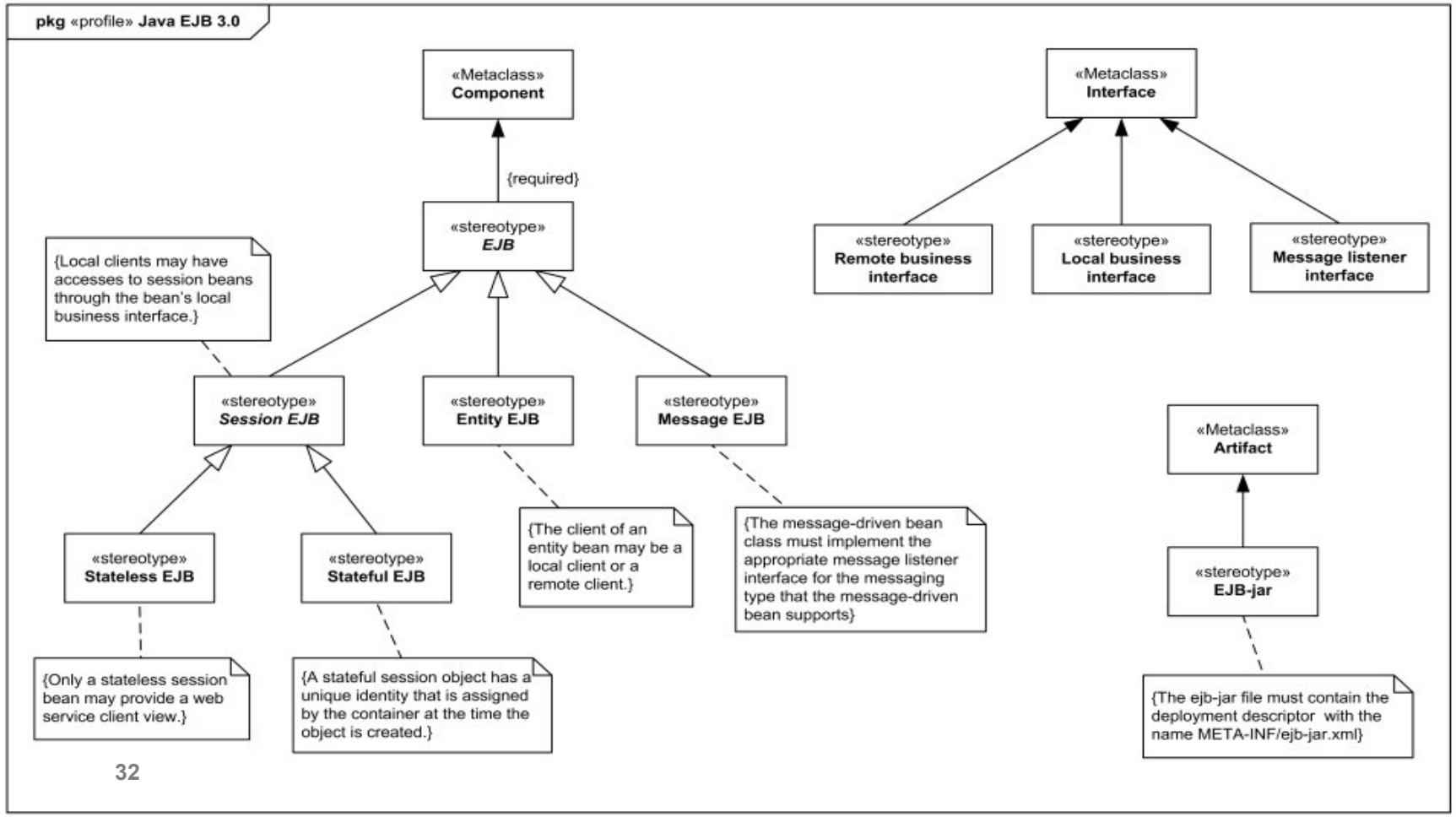
Describes **lightweight extension mechanism** to the UML

Profiles allow adaptation of the UML metamodel for different:

- **platforms** (such as J2EE or .NET), or
- **domains** (such as real-time or business process modeling).

UML Diagrams

- Profile Diagram eg.



Traditional Approach vs OO Approach

Eg. A Library System-Lending Scenario

	Entities
• Librarian enters the borrower id.	
• System checks whether borrower id exist.	**Member**
• Check for Overdue Books.	**Member , LendingDetails**
• Check Over limit	
• Enter copy id	**Member, LendingDetails**
• Check Copy Borrowable	
• Librarian Confirm Borrowing	
• Update Borrowed Copy details	**Copy, Book**

LendingDetails

Traditional Approach vs OO Approach

Eg. A Library System-Lending Scenario

	Classes
• Librarian enters the borrower id.	
• System checks whether borrower id exist.	**Member**
• Check for Overdue Books.	**Member , LendingDetails**
• Check Over limit	
• Enter copy id	
• Check Copy Borrowable	**Member, LendingDetails**
• Librarian Confirm Borrowing	
• Update Borrowed Copy details	**Copy, Book**
	LendingDetails

Understanding OOAD Process (Typical Approach)

- Gather Requirements – What problem you are going to solve
- Describe the application – using actors, use cases and narratives
- Identify the main objects – use narratives
- Describe the Interactions – object interactions , sequence diagrams
- Create the Class Diagram- map objects to classes

Gathering Requirements

- Functional Requirements : What does it do? (Features, Capabilities)

  eg. Program must allow receipts to be generated by e-mail

- Non-Functional Requirements : What else?
  - Legal (eg. Banking System laws one needs to comply with)
  - Performance Requirements : eg Response time etc. Respond to requests in 2 seconds.

An Introduction to Use-Case Modeling

- The process of modeling a system's functions in terms of
  - business events
  - who initiated the events
  - how the system responds to those events
- An approach that facilitates ***user-centered development***

User-Centered Development and Use-Case Modeling

User-centered development – a process of systems development based on understanding the ***needs of the stakeholders*** and the ***reasons*** why the system should be developed.

Use-case modeling – the process of modeling a system's functions in terms of business events, who initiated the events, and how the system responds to those events.

- Use-case modeling has roots in object-oriented modeling.
- Popular in non-object development environments because of its usefulness in communicating with users.

Sequence Diagrams

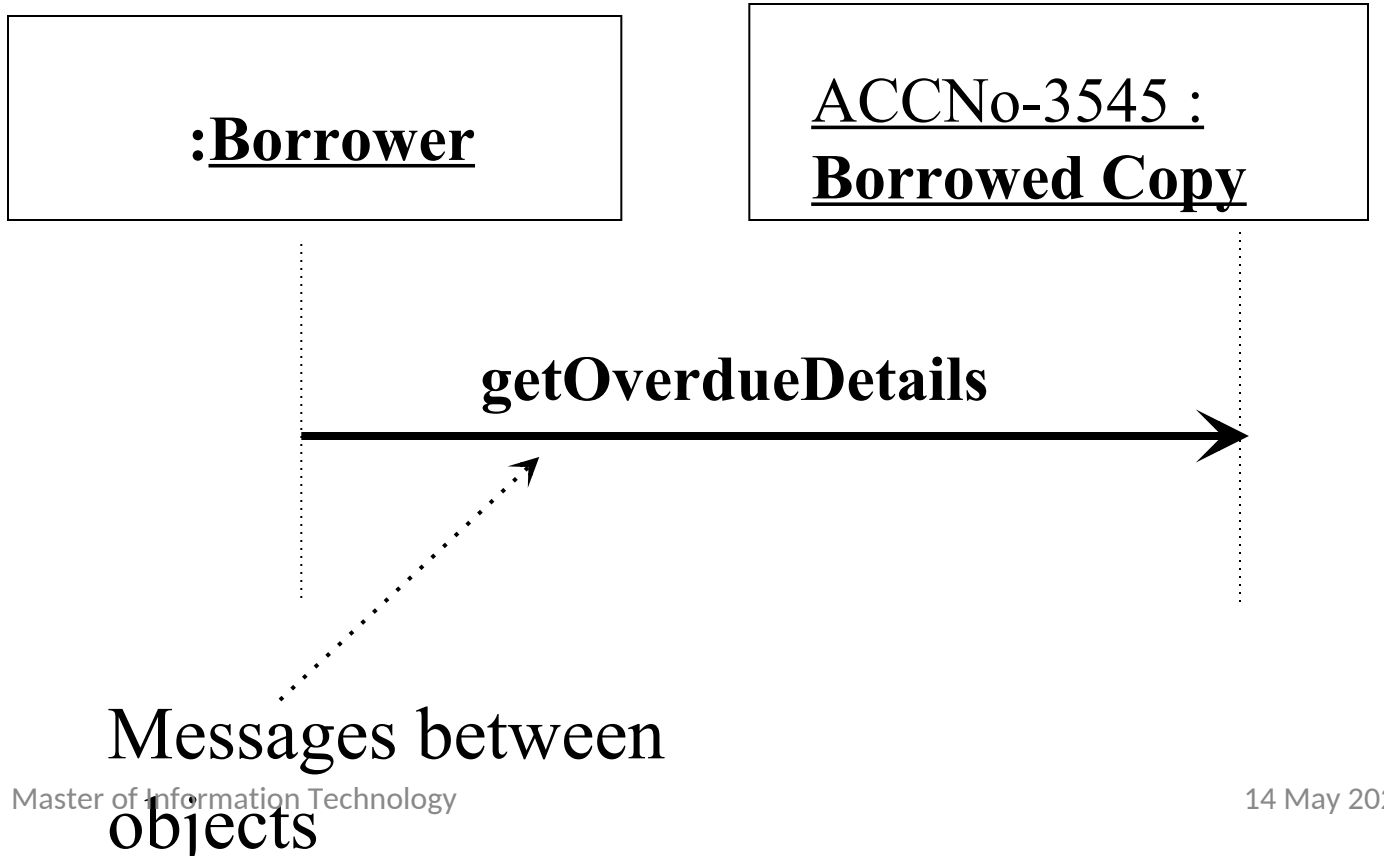
- Shows object interactions arranged in time sequence.
- Shows the objects and classes involved in the scenario.
- Shows the sequence of messages exchanged between the objects needed to carry out the functionality of the scenario.

Sequence Diagrams cont..

- In UML, an object in a sequence diagram is drawn as a rectangle, containing the name of the object, underlined.
- An object can be named in one of the three ways:
  - Object name,
  - Object name and its class,
  - Class name (anonymous object)

Sequence Diagrams cont..

- UML Notation for objects and messages in a sequence diagrams is shown below:



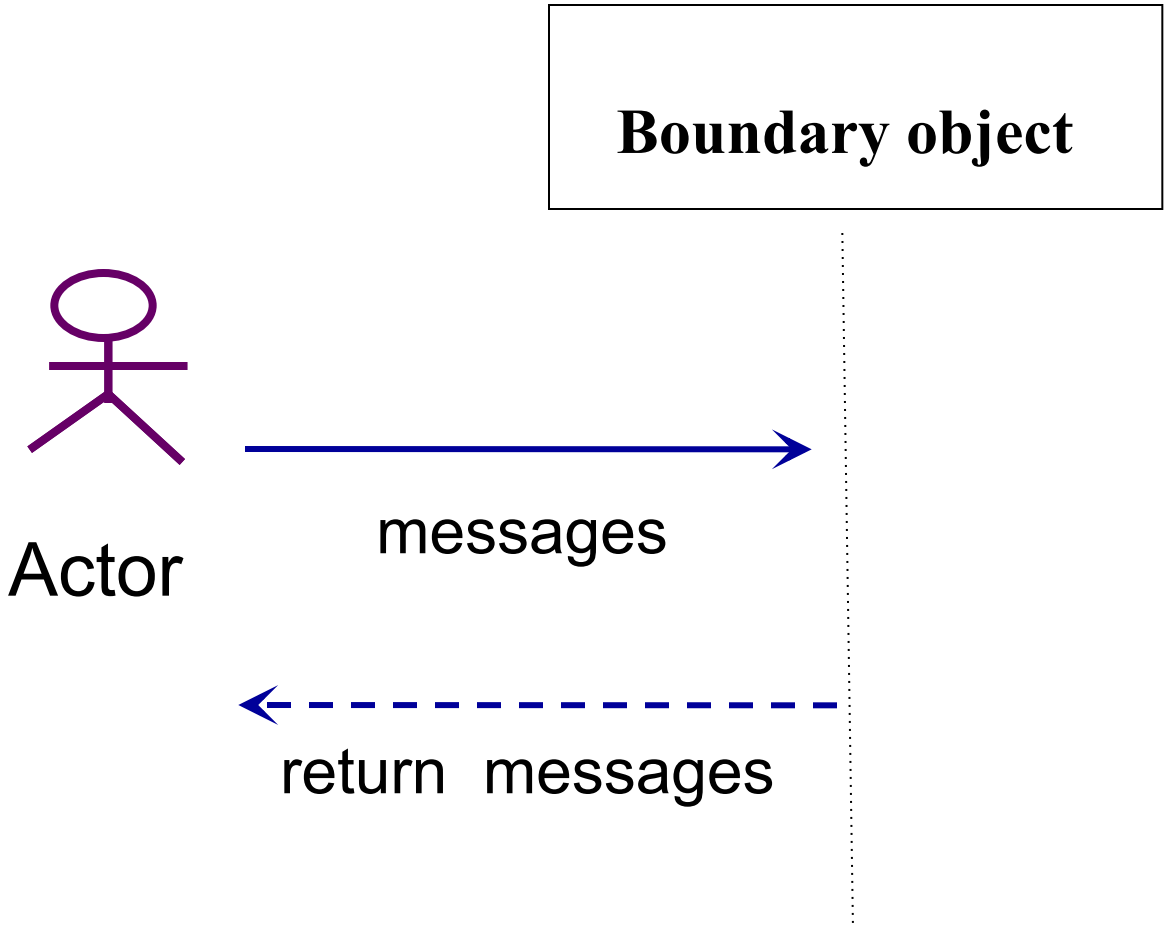
Sequence Diagrams and Boundary classes

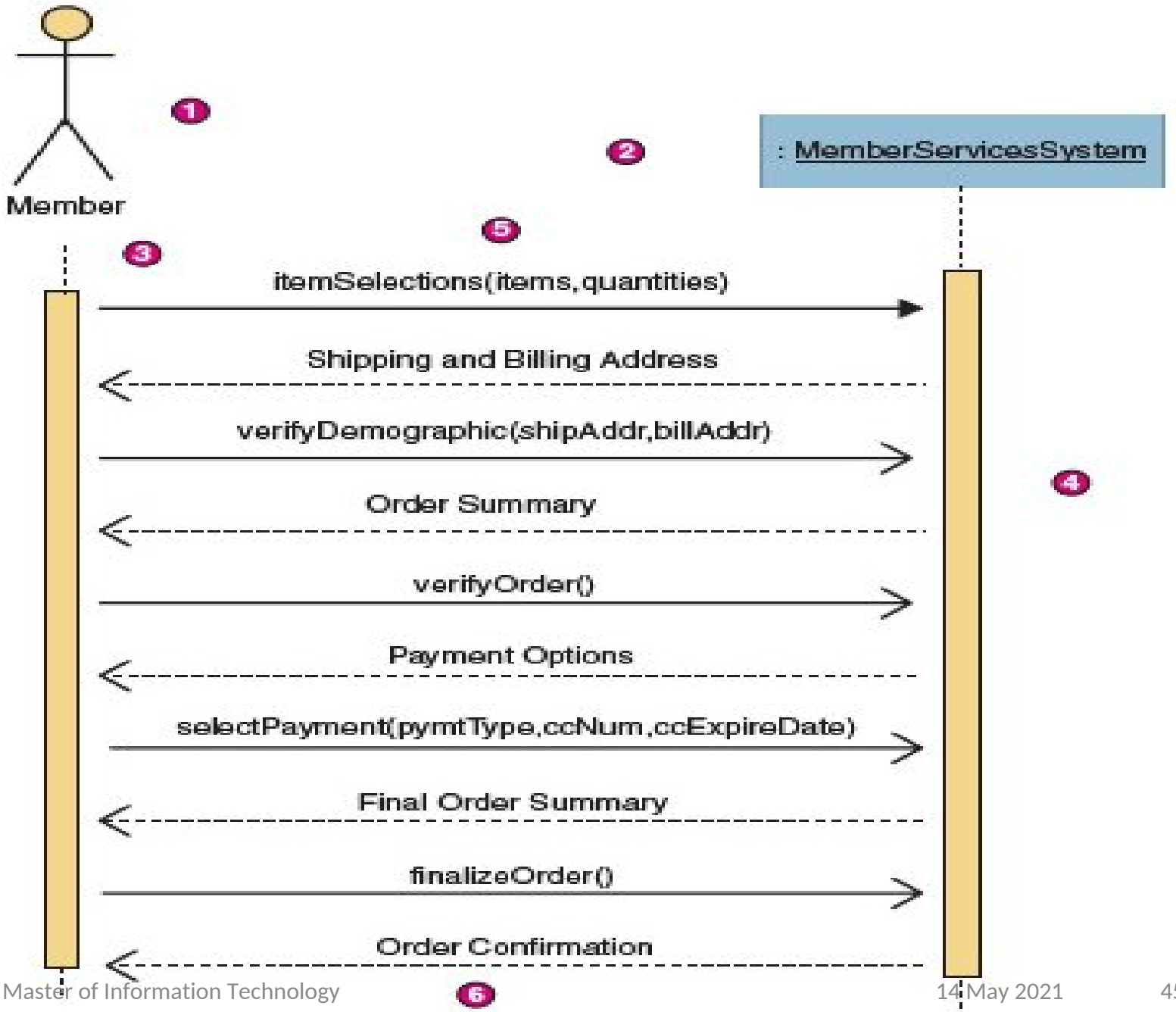
- Boundary classes are added to sequence diagrams to show the interaction with the user or another system.
- During the early analysis phases, boundary classes are shown on a sequence diagram only to capture and document the interface requirements.
- Actual messages from the actor to boundary class with their sequencing information will depend on the application framework that will be selected later in development.

System Sequence Diagrams

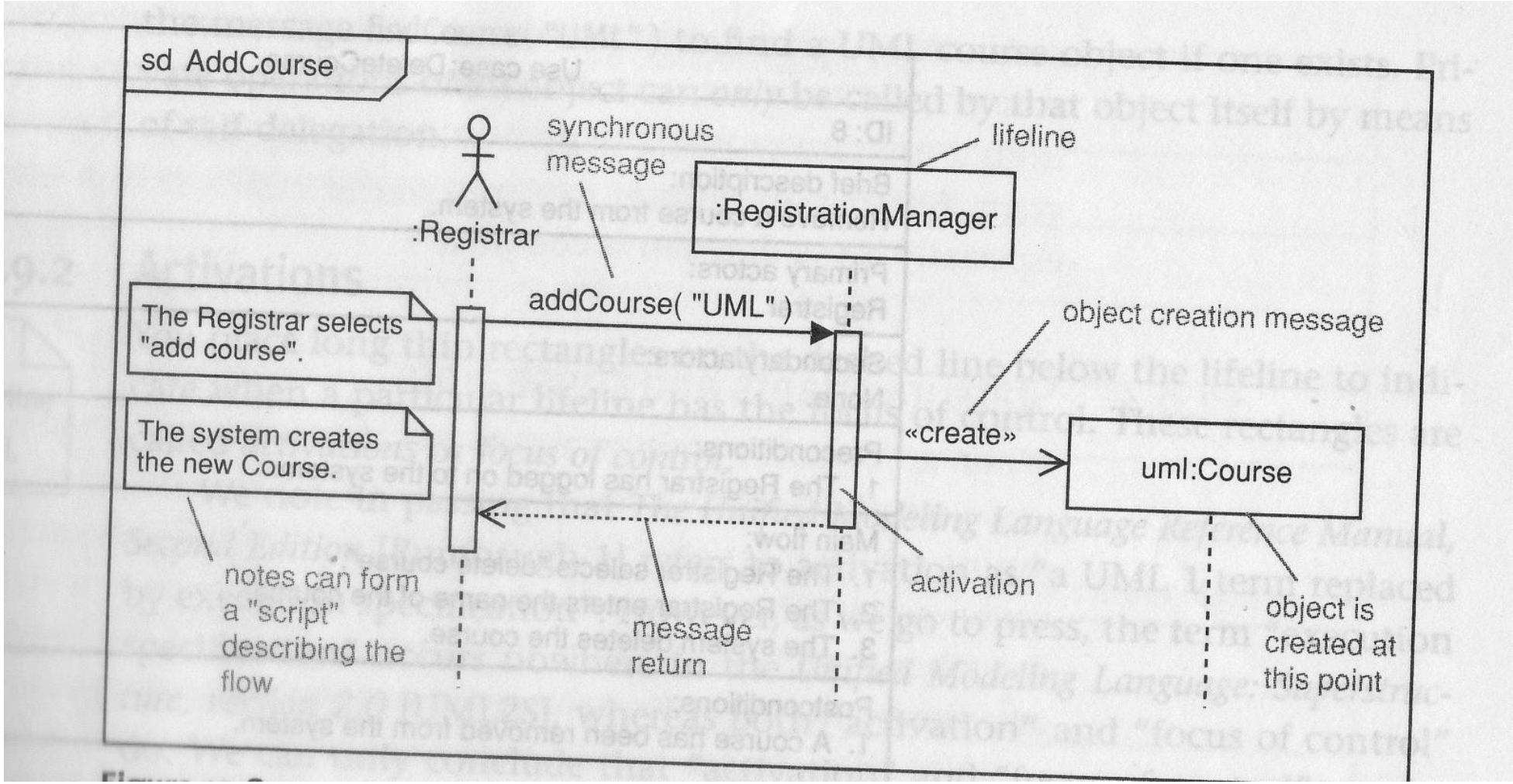
- A tool used by some Analysts in logical design phase is the system sequence diagram.
- It helps us to identify the high level messages that enter and exit the system.
- Later these messages will become the responsibility of individual objects.
- These individual objects will fulfill those responsibilities by communicating with other objects.

System Sequence Diagrams




Member
: MemberServicesSystem
1
2
3
4
5
6
itemSelections(items,quantities)
Shipping and Billing Address
verifyDemographic(shipAddr,billAddr)
Order Summary
verifyOrder()
Payment Options
selectPayment(pymtType,ccNum,ccExpireDate)
Final Order Summary
finalizeOrder()
Order Confirmation

Eg. Sequence Diagram: Add Details of a new Course



Framing in Sequence Diagrams

- UML 2.* feature
- One can frame a sequence diagram by surrounding it with a border and adding a compartment in the upper left corner.
- The compartment contains information that identifies the diagram.
- These interaction fragments can be combined.
- Gives you a quick and easy way to reuse part of one sequence diagram in another.
- Ref. http://www.youtube.com/watch?v=4WDbte6cPa8

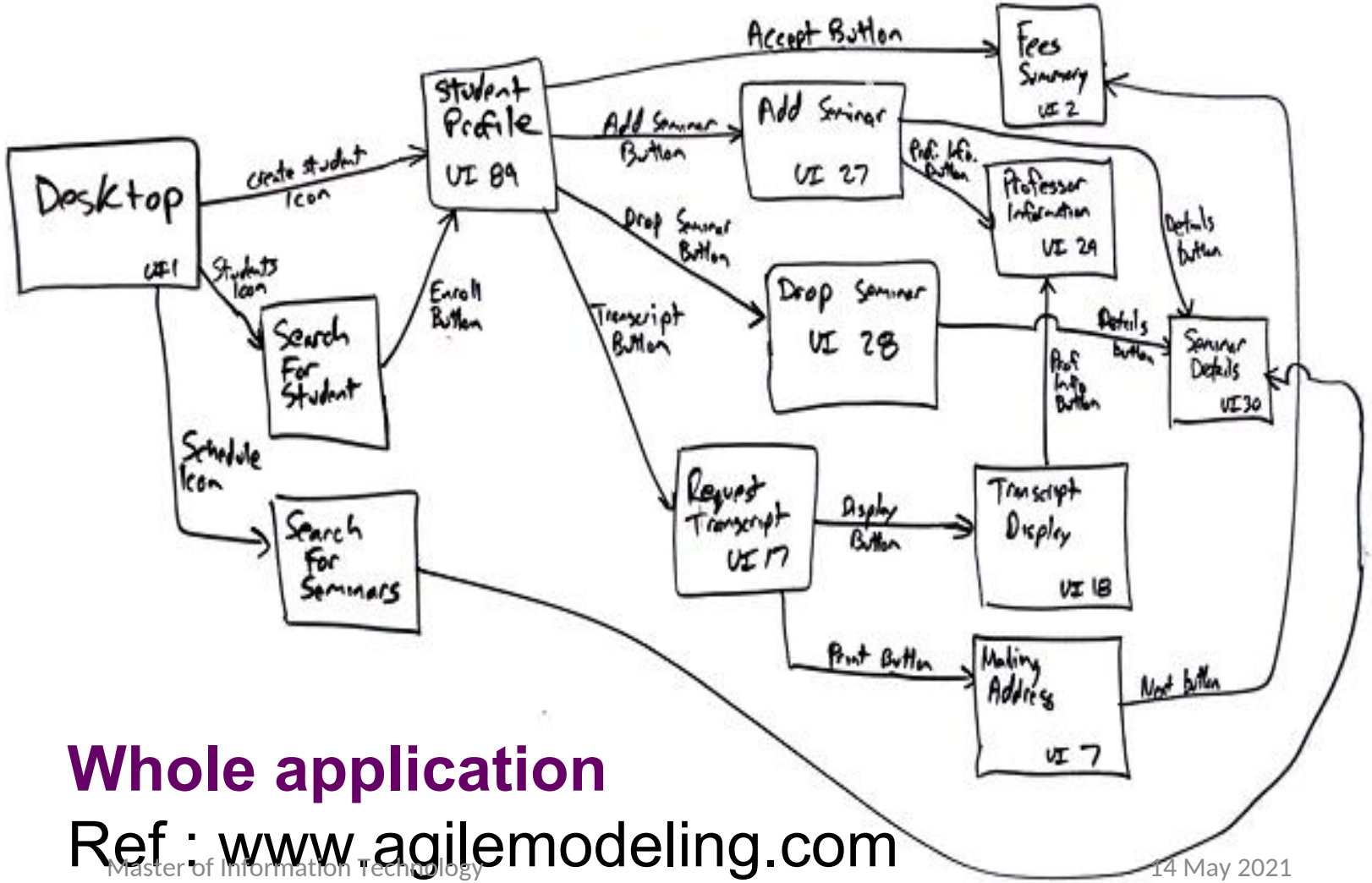
User Interface Flow Diagrams (UI Storyboards)

- Not part of UML
- Enable you to model the high-level relationships between major user interface elements.
- The **boxes** represent major user interface elements, modeled as you would instances/objects, and the **arrows** represent the possible flow between them, modeled as you would transitions in activity diagrams.

User Interface Flow Diagrams

- You can draw it for the whole application (A) or for a single Use Case Scenario (B).
- (A) enable you to gain a high-level overview of the user interface for your application.
- This overview is effectively the combination of all the behavioral views derived from your use case

User Interface Flow Diagrams



Whole application

Ref : www.agilemodeling.com

User Interface Flow Diagram for a **Single Use Case Scenario (B).**

Exercise : Draw a User Interface Flow Diagram for Borrowing Use Case

Steps: (Flow of Events taken from Use Case Narrative)

Librarian enters the borrower id.

System checks whether borrower id exist.

If not exist (E-1) end use case.

Else **Process**

Check for Overdue Books.

If yes (E-2) end use case

Check Over limit (E-3)

If yes (E-3) end use case Continue….

Draw a User Interface Flow Diagram for Borrowing Use Case - continue…

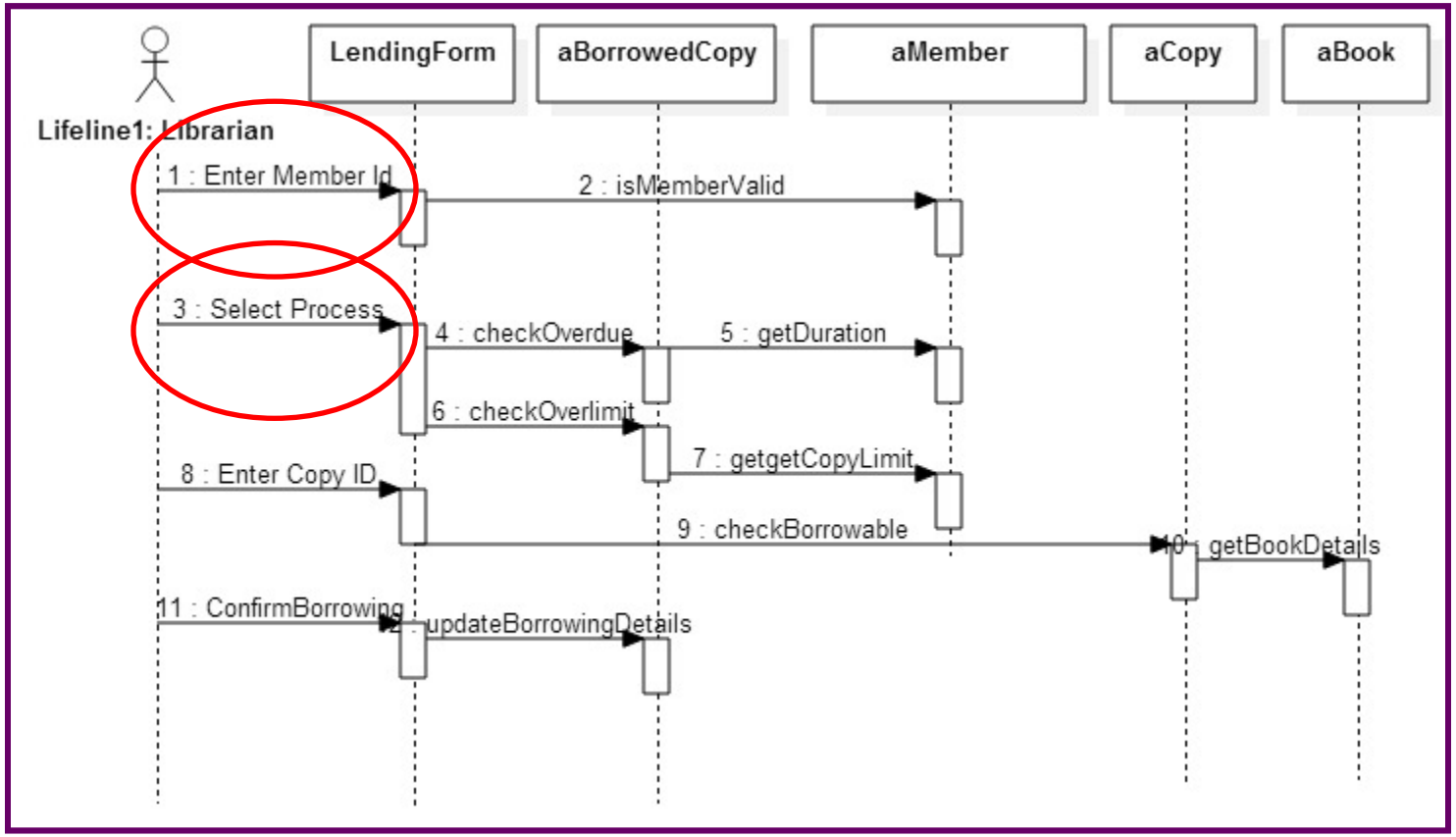
Enter copy id
Check Borrow able (E-4)
    If No (E-4) end use case
Librarian Confirm Borrowing (C-5)
Update Borrowed Copy details

Alternate Flows

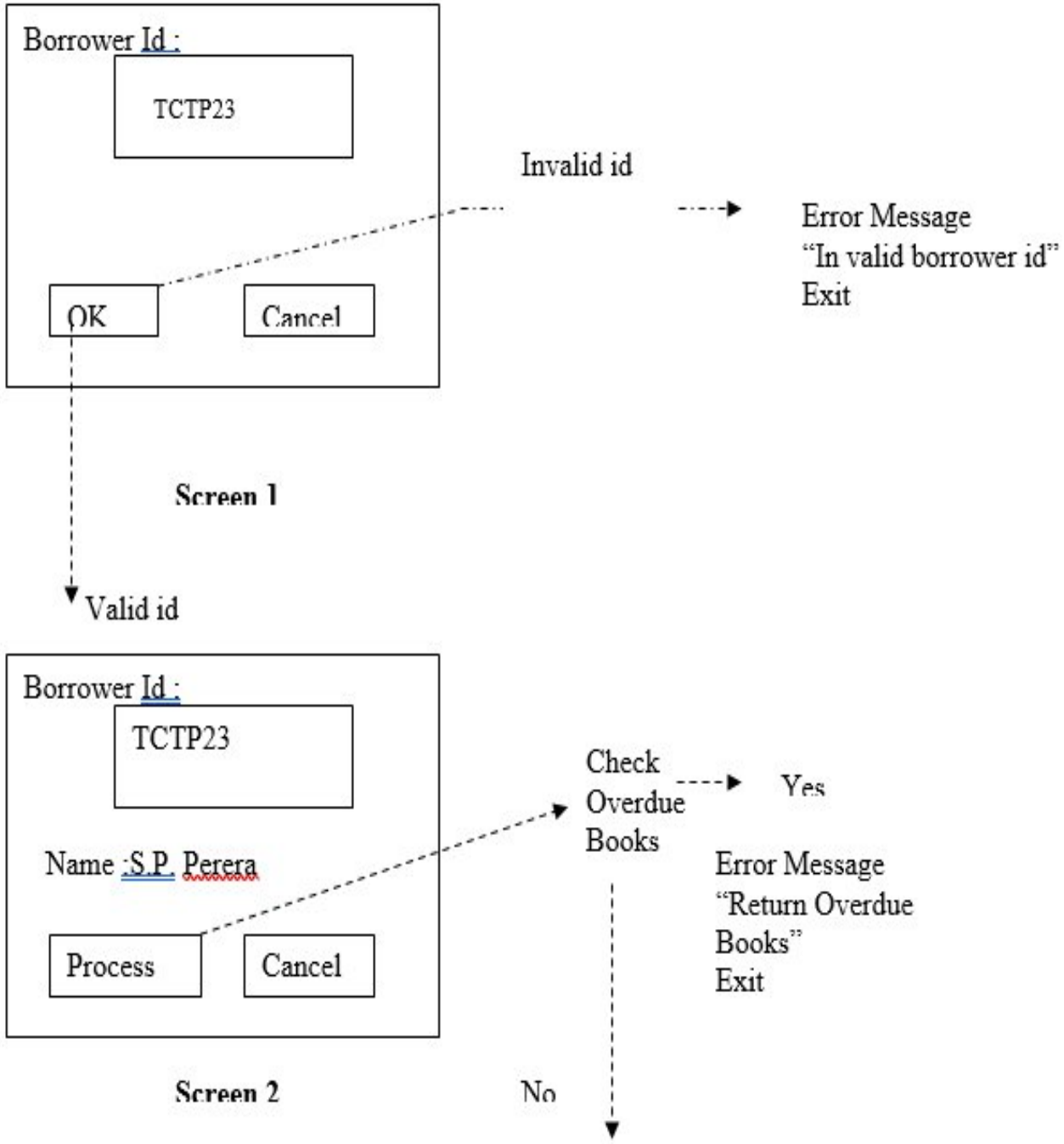
E-1 : Borrower id does not exist
E-2 : There are overdue books
E-3 : Borrower has already borrowed 5 books (max)
E-4 : Copy is not borrow able
C-1 :   Confirm borrowing Message box

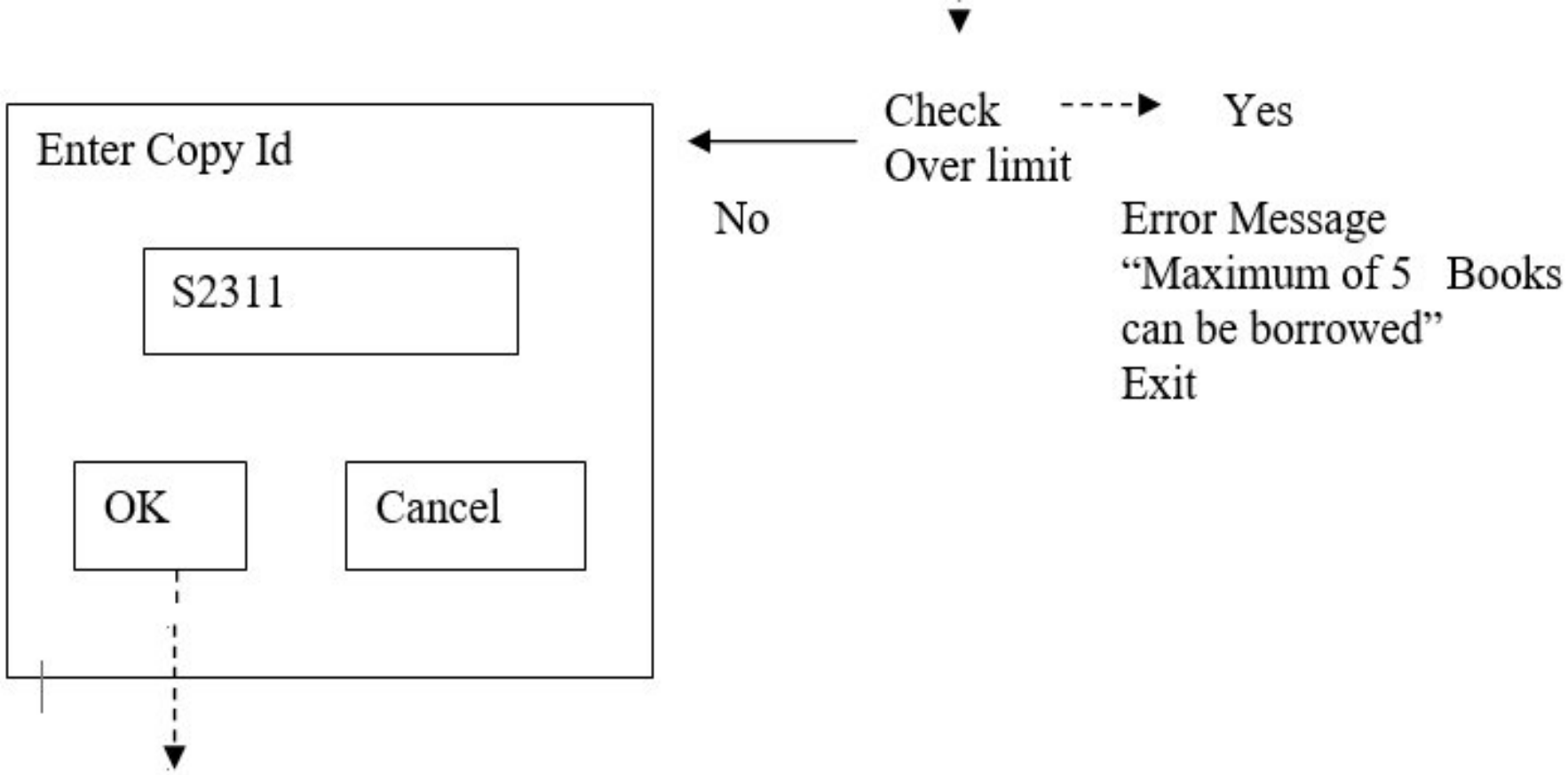
User Interface Flow Diagrams

- Also called storyboards, interface-flow diagrams, windows navigation diagrams, and context-navigation maps
- See more at: http://agilemodeling.com

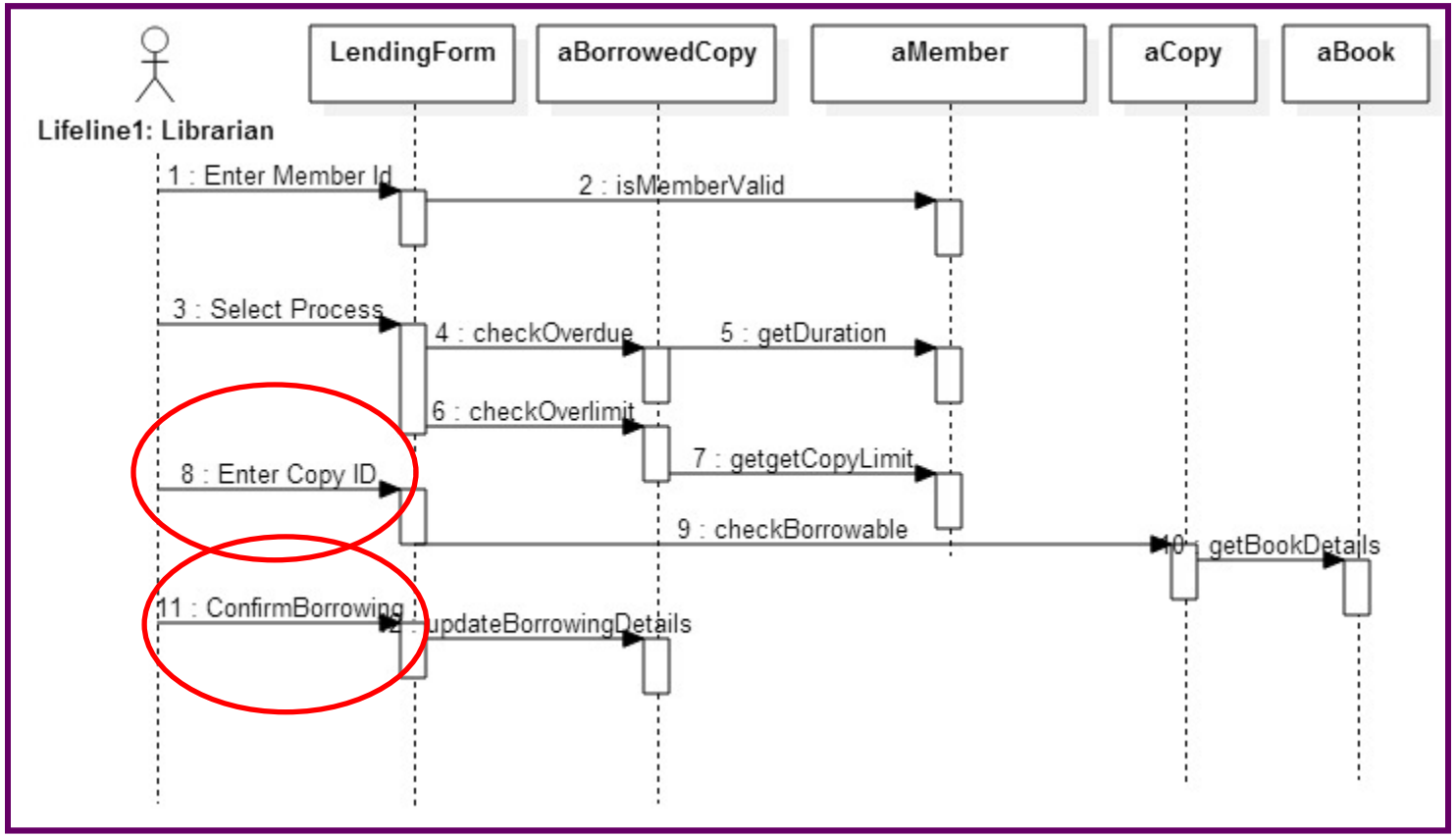


Borrowing Sequence Diagram with objects and messages


Borrower Id :
TCTP23
OK
Cancel
Invalid id
Error Message
"In valid borrower id"
Exit
Screen 1
Valid id
Borrower Id :
TCTP23
Name :S.P. Perera
Process
Cancel
Check
Overdue
Books
Yes
Error Message
"Return Overdue
Books"
Exit
No
Screen 2


Check
Over limit
Yes
No
Error Message
"Maximum of 5 Books
can be borrowed"
Exit
Enter Copy Id
S2311
OK
Cancel

Screen 3



Borrowing Sequence Diagram with objects and messages

Use Case Narratives cont..

Eg. Borrowing Scenario Flow of Events

Main Flow

Librarian enters the borrower id.

System checks whether borrower id exist.

  If not exist (E-1) end use case.

Else **Process**

Check for Overdue Books.

  If yes (E-2) end use case

Check Over limit (E-3)

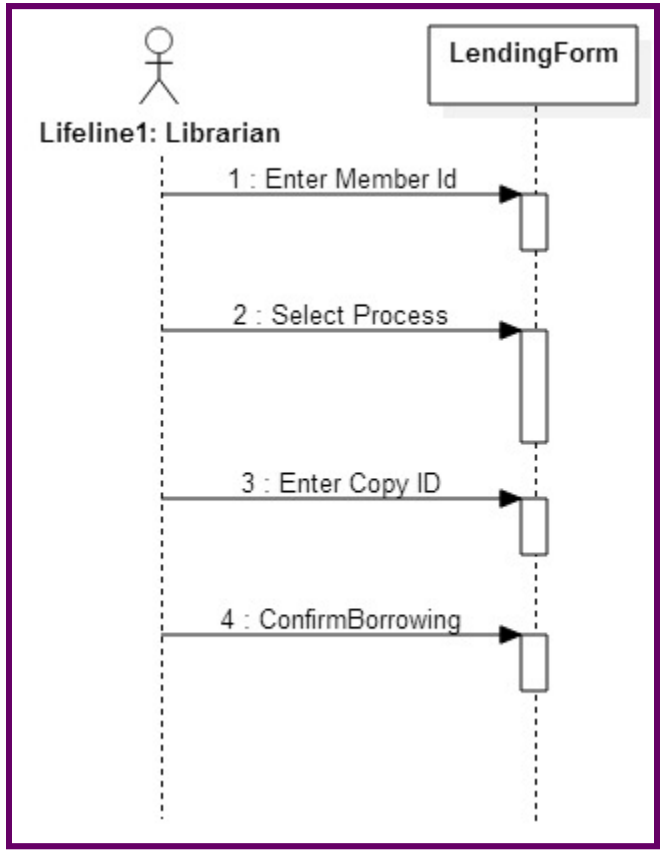
  If yes (E-3) end use case

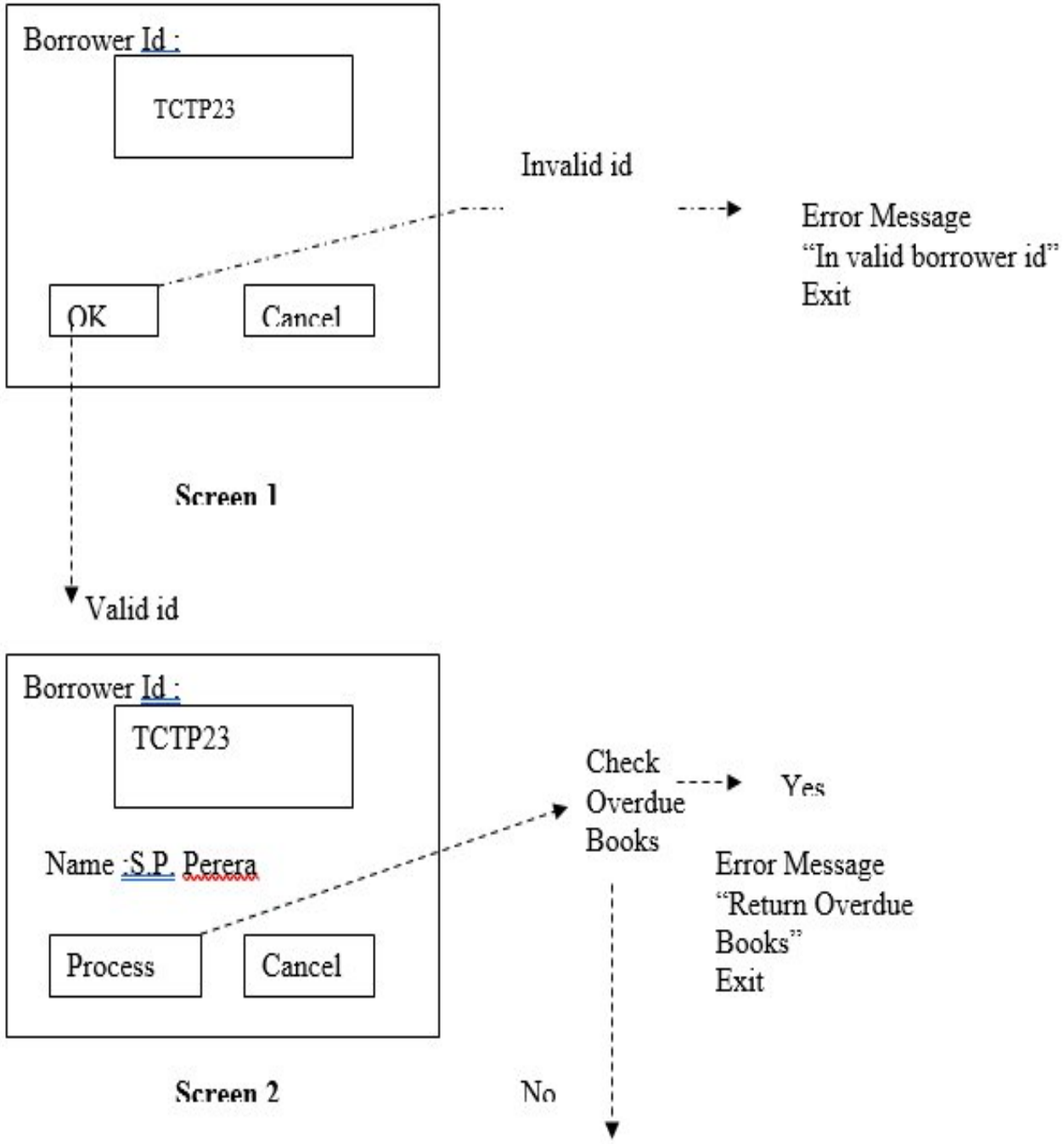
Use Case Narratives cont..

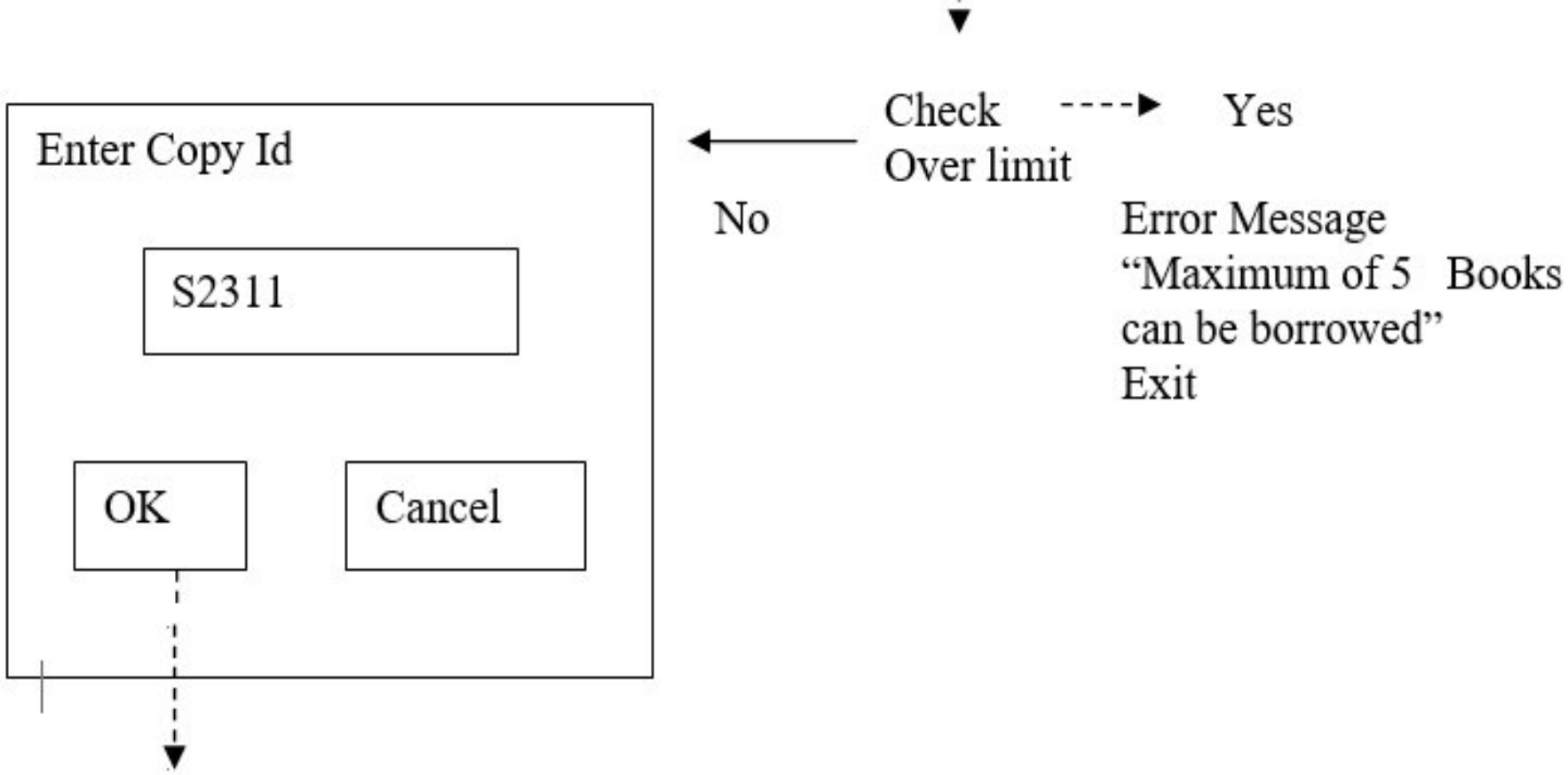
Enter copy id
Check Borrow able (E-4)
    If No (E-4) end use case
Librarian Confirm Borrowing (C-5)
Update Borrowed Copy details

Alternate Flows
E-1 : Borrower id exist
E-2 : There are overdue books
E-3 : Borrower has already borrowed 5 books (max)
E-4 : Copy is not borrow able
C-1 :   Confirm borrowing Message box

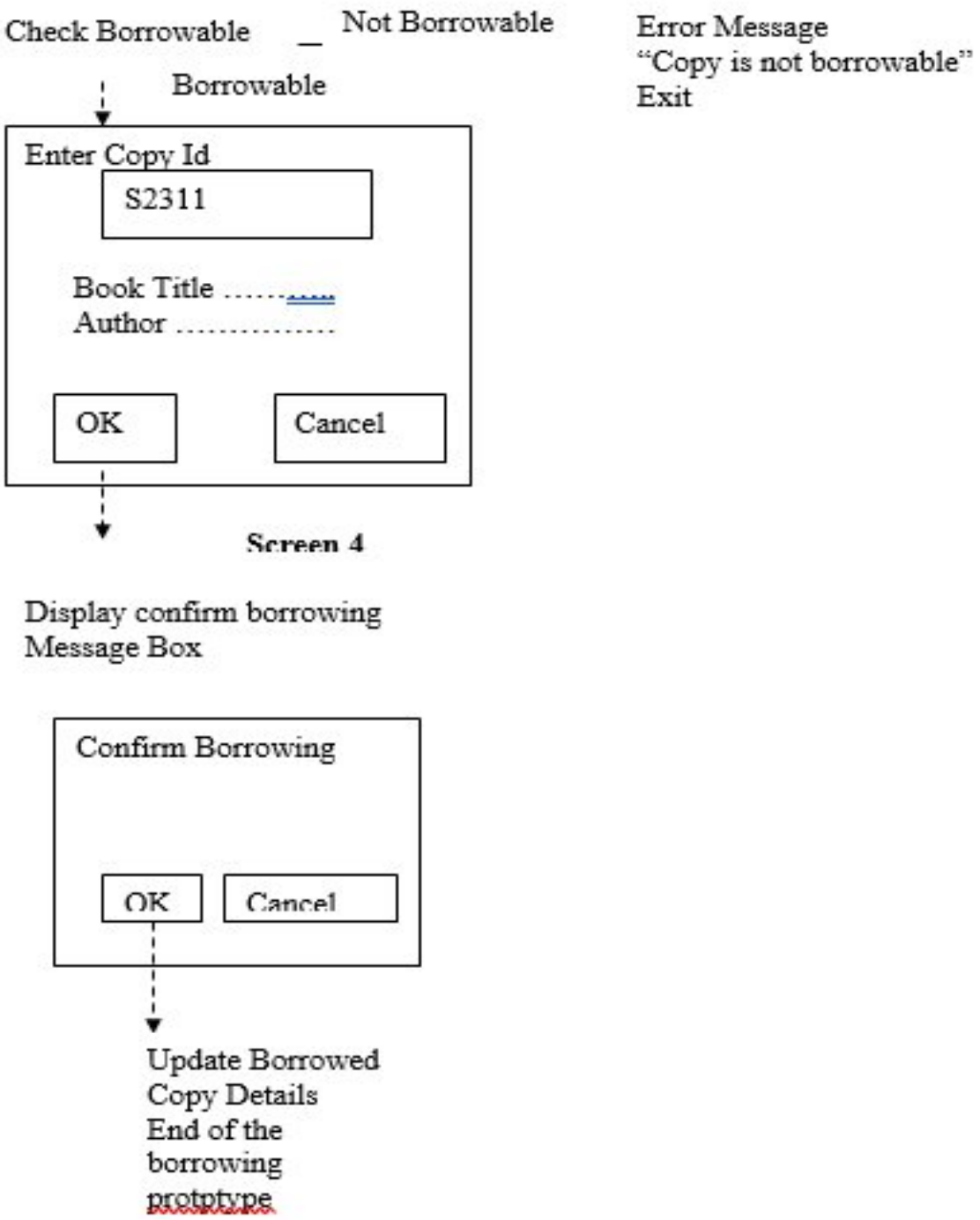
System Sequence Diagram for Borrowing Scenario




Borrower Id :
TCTP23
OK
Cancel
Invalid id
Error Message
"In valid borrower id"
Exit
Screen 1
Valid id
Borrower Id :
TCTP23
Name :S.P. Perera
Process
Cancel
Check
Overdue
Books
Yes
Error Message
"Return Overdue
Books"
Exit
No
Screen 2


Check
Over limit
Yes
No
Error Message
"Maximum of 5 Books
can be borrowed"
Exit
Enter Copy Id
S2311
OK
Cancel

Screen 3


Check Borrowable
Not Borrowable
Error Message
"Copy is not borrowable"
Exit
Borrowable
Enter Copy Id
S2311
Book Title .....
Author .....
OK
Cancel
Screen 4
Display confirm borrowing
Message Box
Confirm Borrowing
OK
Cancel
Update Borrowed
Copy Details
End of the
borrowing
protptype

General activities in performing OOA

- Modeling the functions of the system

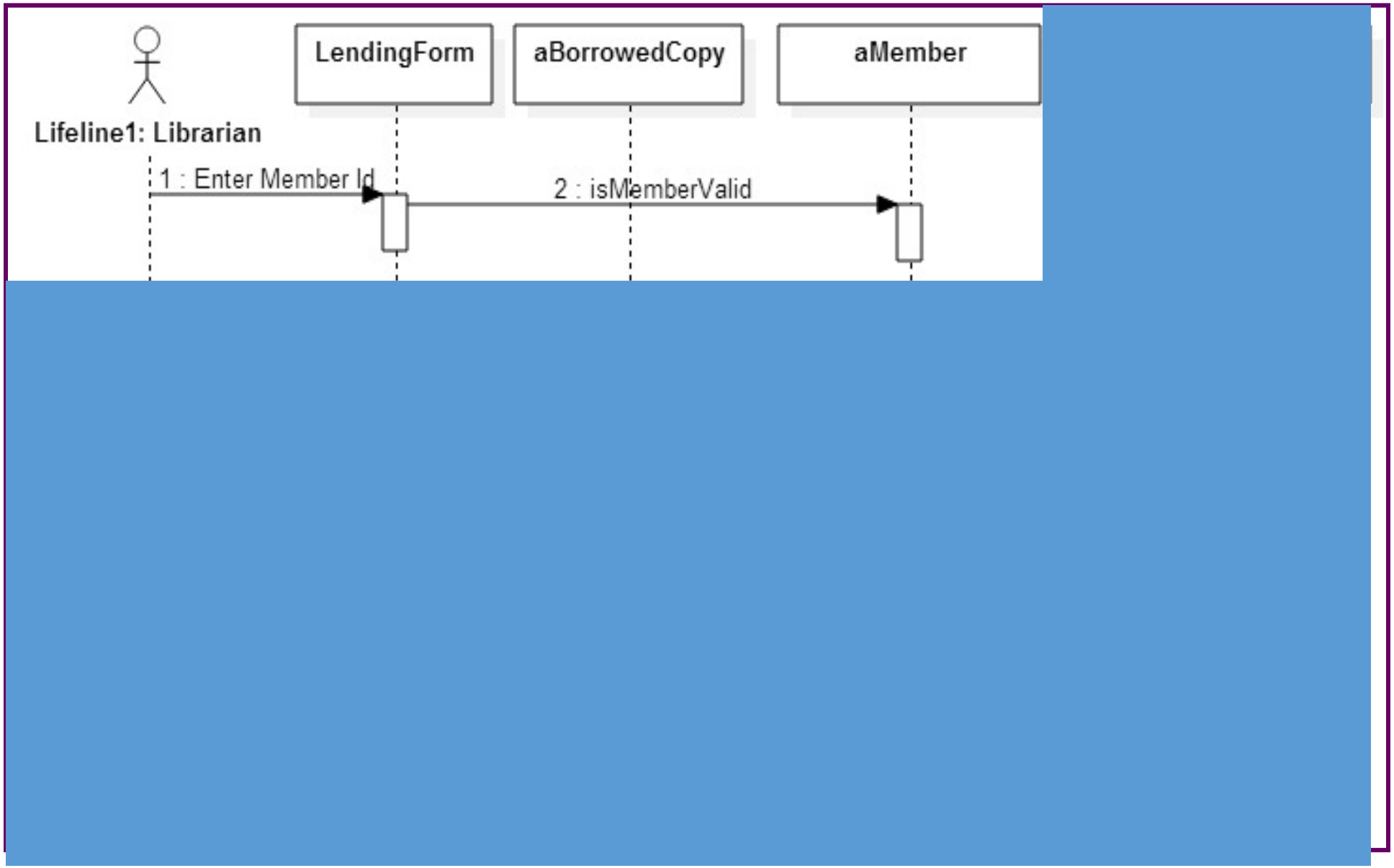
  Refine the requirements use case model and use case narratives, Model the use case activities (draw activity diagrams), Draw **system sequence diagram** ,

- Finding and Identifying the Business Objects

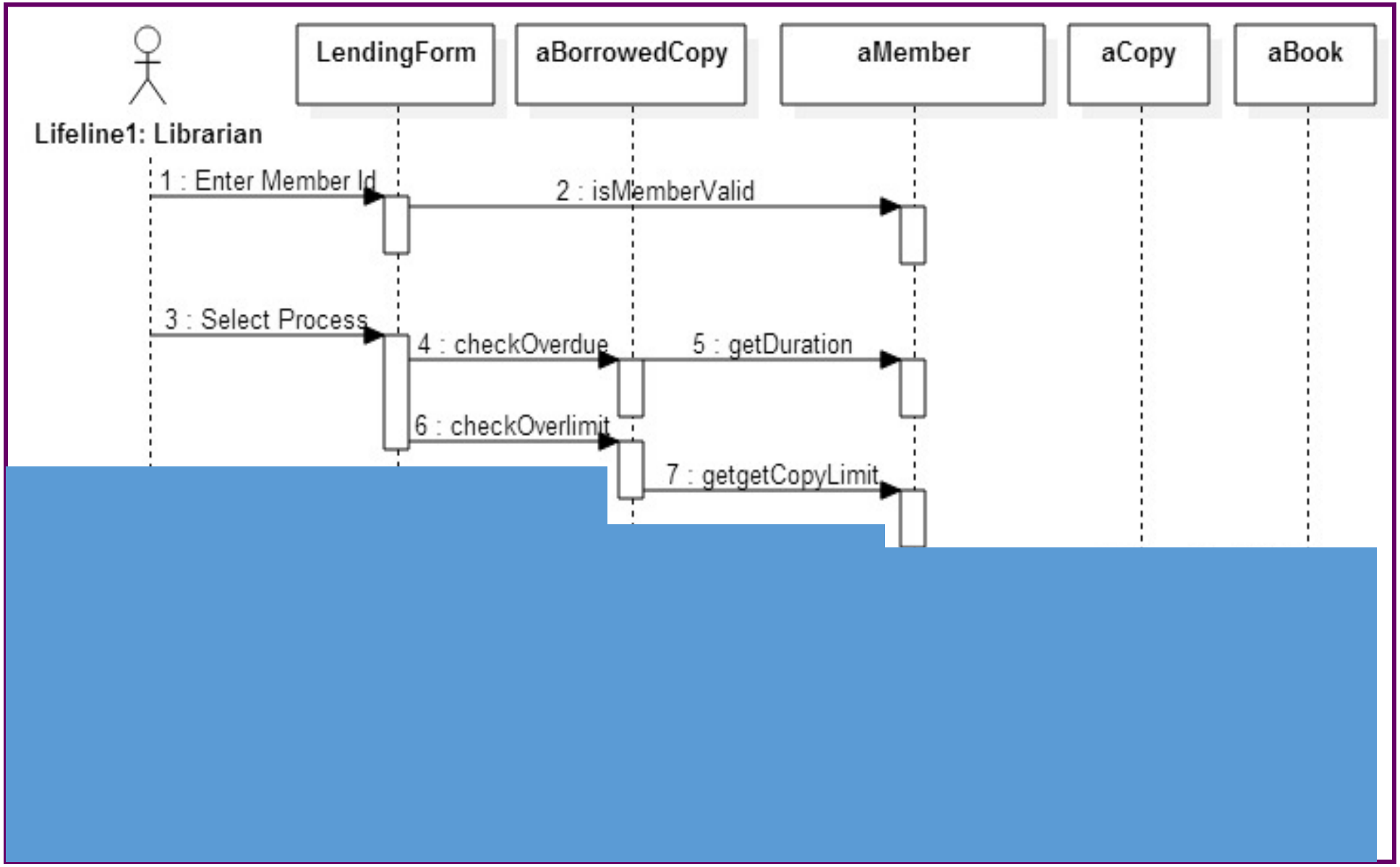
  Noun analysis, CRC analysis

- Organizing the Objects and Identifying the Relationships

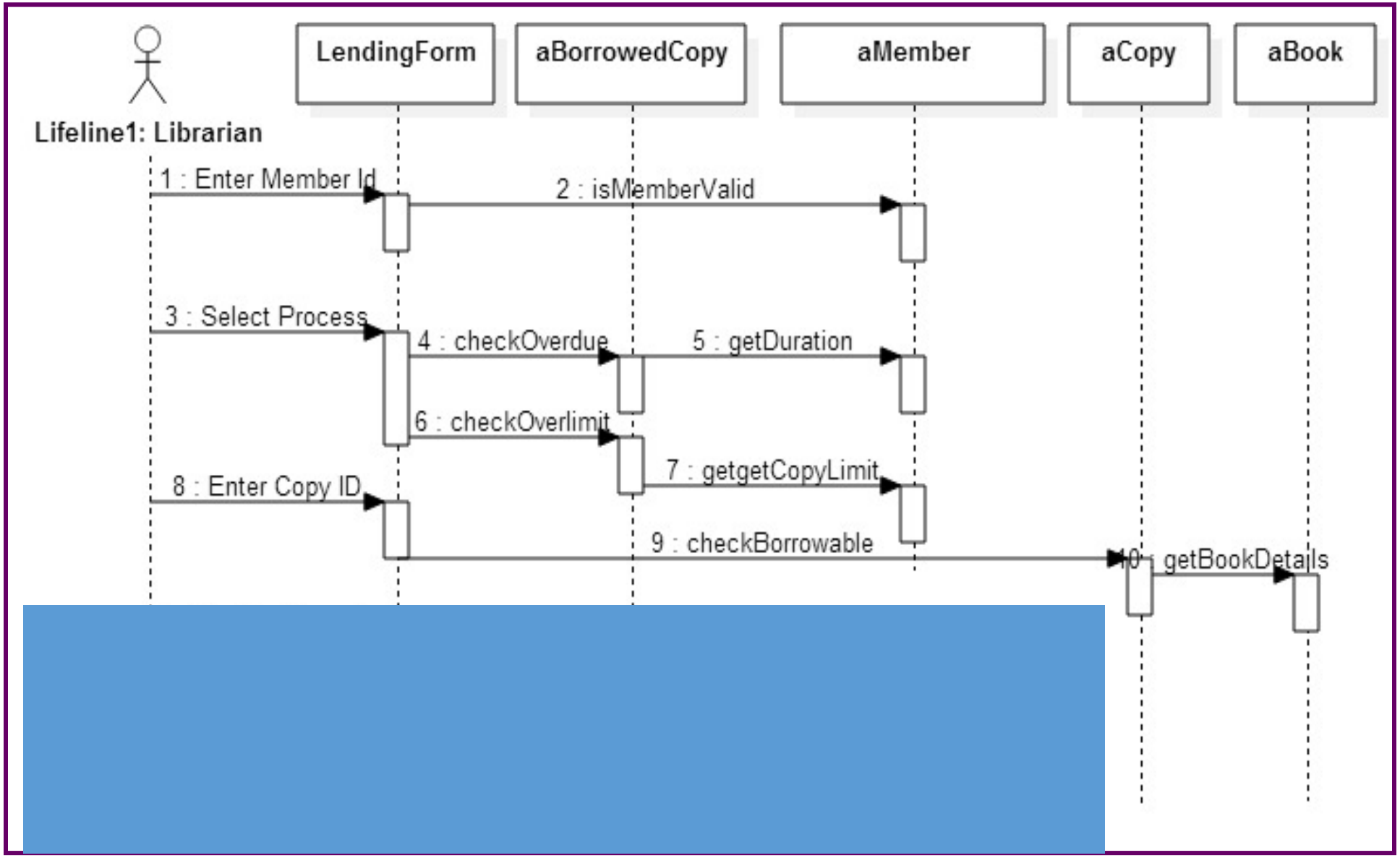
  Identify types of relationships between the objects. Class diagram with problem domain classes.



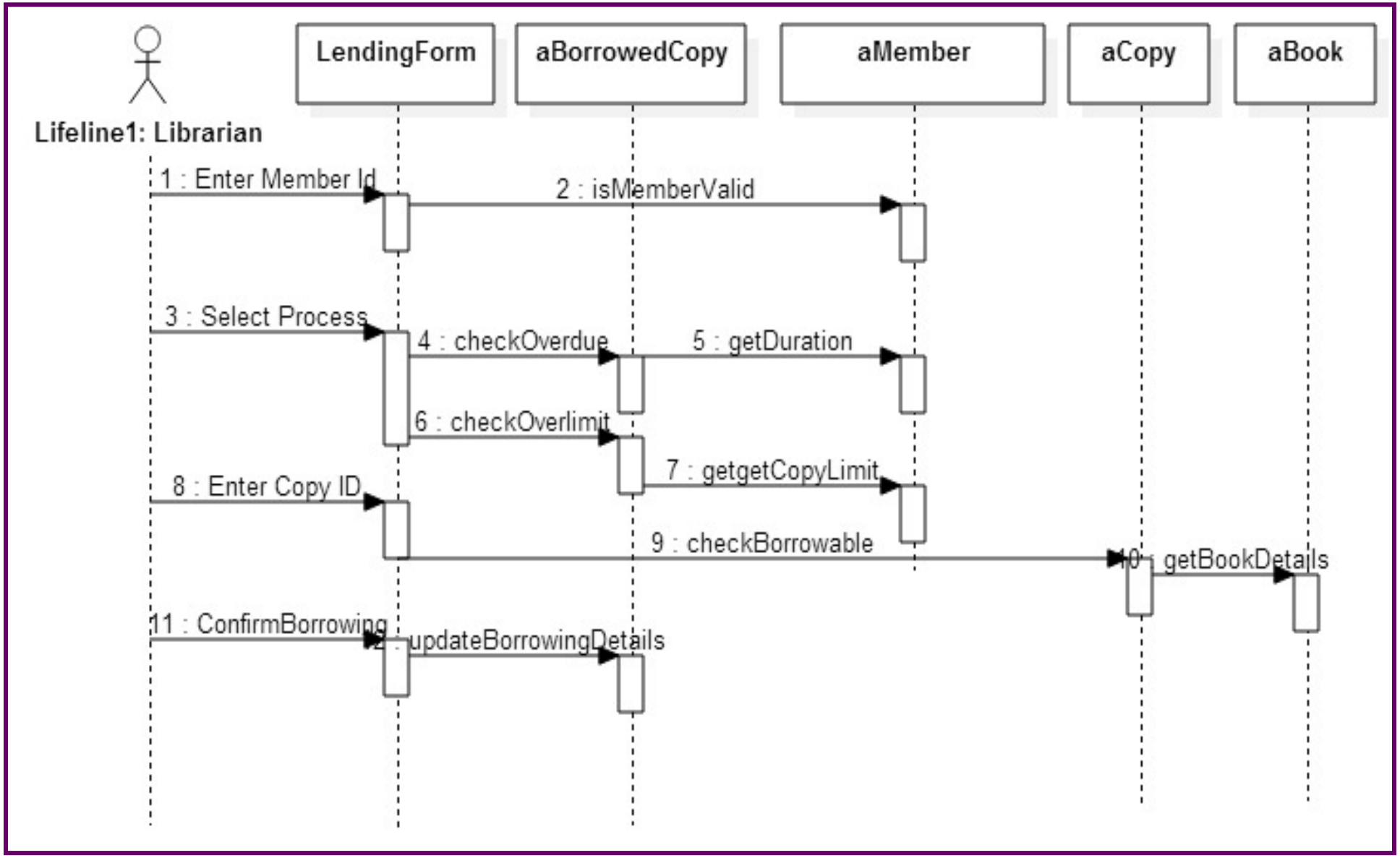
Borrowing Sequence Diagram with objects and messages



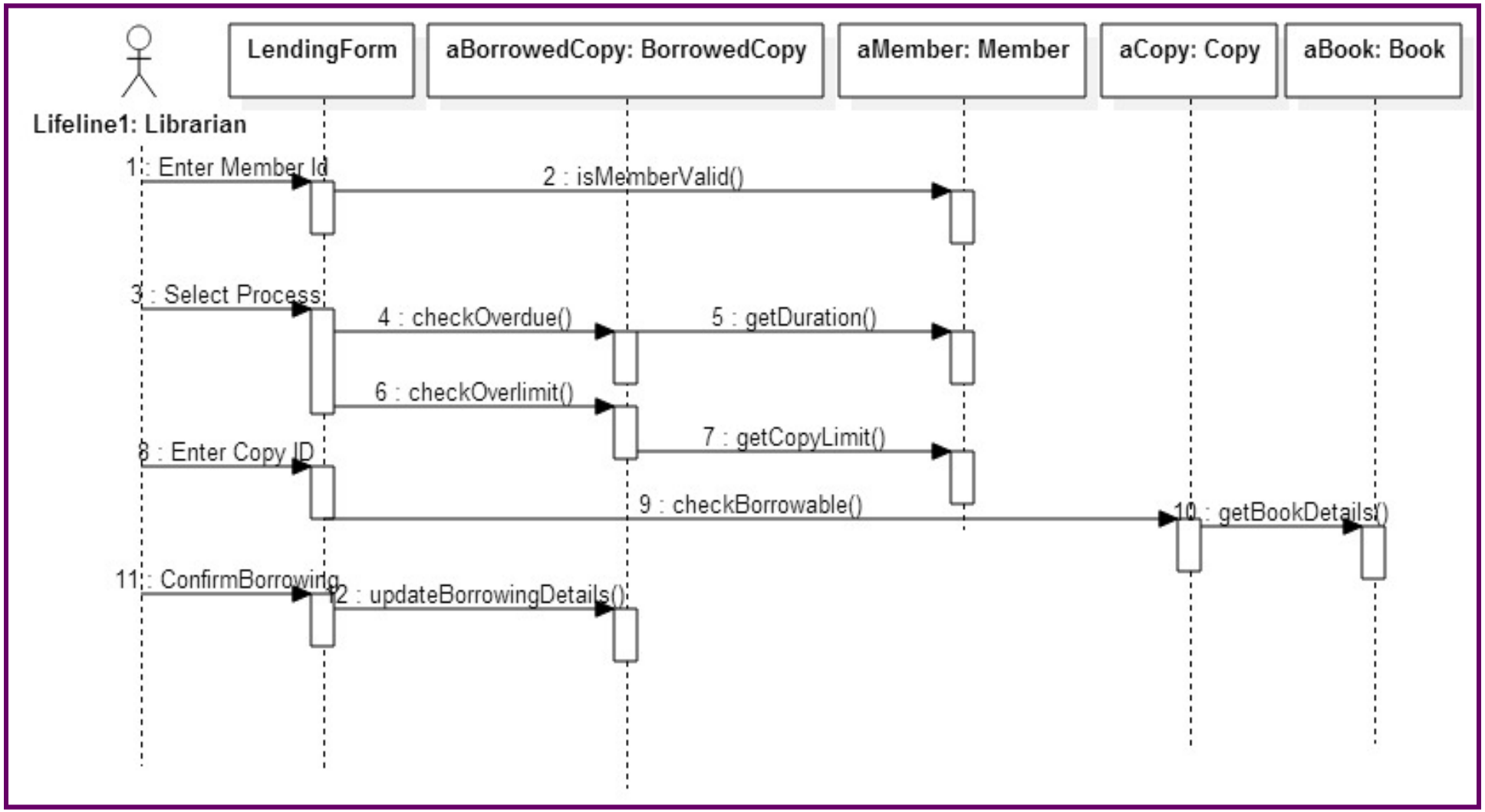
Borrowing Sequence Diagram with objects and messages



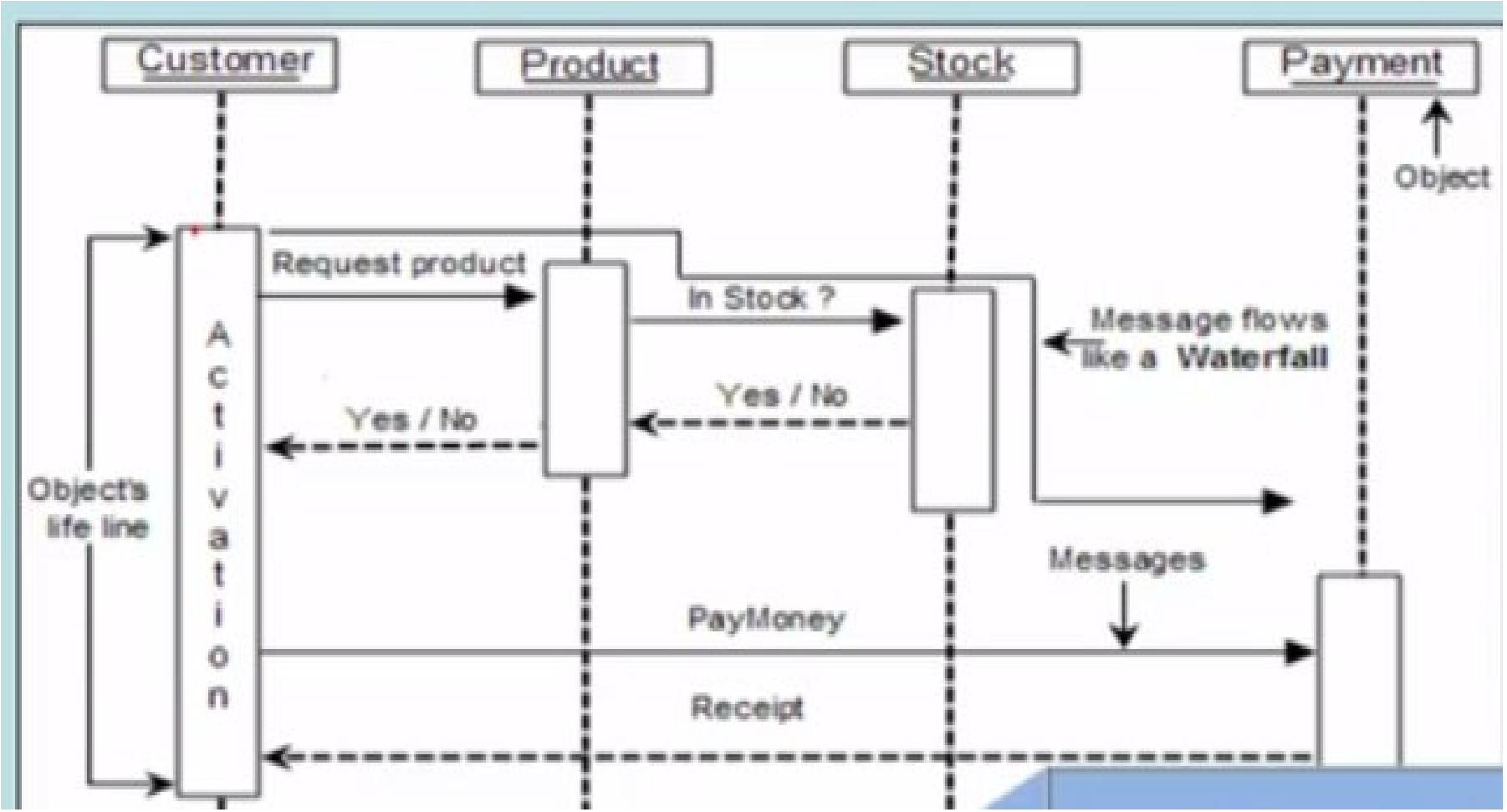
Borrowing Sequence Diagram with objects and messages



Borrowing Sequence Diagram with objects and messages



Borrowing Sequence Diagram

After mapping objects to classes and messages to methods

Another Example